Contents

The DevOps Pipeline Playbook **4**
This playbook has a companion . . . 4
Table of Contents . . . 5
Philosophy . . . 5
Choose your track . . . 6
Track A — Self-hosted (VPS + Docker + blue-green) . . . 6
Track B — Platform-managed (PaaS + BaaS + atomic deploys) . . . 7
A short decision aid (not a verdict) . . . 7
Pipeline at a Glance . . . 8
1. The full release chain — commit to production . . . 8
2. CI job graph — what runs on every push and PR . . . 8
3. Blue-green deploy — what happens on the server . . . 9
Prerequisites . . . 11

Track A — Self-hosted (VPS + Docker + blue-green) **11**
Phase 0: Foundation (Before Anything Else) . . . 12
0.1 Git-flow branching . . . 12
0.2 Conventional commits + commitlint . . . 12
0.3 .gitignore and .dockerignore . . . 13
0.4 The VERSION file . . . 14
0.5 First commit to main . . . 14
[OK] Phase 0 verification . . . 14
Phase 1: Local Development . . . 15
1.1 Multi-stage Dockerfiles . . . 15
1.2 docker-compose.yml for local dev . . . 16
1.3 Production compose file . . . 17
1.4 .env.example . . . 18
[OK] Phase 1 verification . . . 18
Phase 2: Basic CI . . . 19
2.1 The skeleton . . . 19
2.2 Commit Message Lint . . . 19
2.3 Backend tests . . . 20
2.4 Frontend tests . . . 21
2.4a Performance and accessibility budgets — Lighthouse CI . . . 22
2.4b Accessibility tests — axe-core in E2E . . . 23
2.4c E2E runs on at least one touch device profile . . . 24
2.4d Rate-limit every public endpoint, not just the authenticated ones . . . 25
2.4e Body size limits on POST and PUT . . . 25
2.5 Secrets scanning . . . 25
[OK] Phase 2 verification . . . 25
Phase 3: CI Self-Protection . . . 26
3.1 Workflow Lint (actionlint) — FIRST JOB . . . 26
3.2 Fix the findings . . . 26
3.3 Docker Build Validation . . . 27
3.4 Container Vulnerability Scan . . . 27
3.5 CI Failure Notification (main-only) . . . 28
[OK] Phase 3 verification . . . 29
Phase 4: Blue-Green Deploy . . . 29
4.1 The shape of the deploy . . . 29
4.2 The deploy script . . . 29
4.3 The smoke test script . . . 33
4.4 Preflight check . . . 35

4.5 Nginx upstream config . . . 35
[OK] Phase 4 verification . . . 36
Phase 5: Automated Release . . . 36
5.1 release-please config . . . 37
5.2 release.yml workflow . . . 37
5.3 deploy.yml — the production deploy trigger . . . 38
5.4 deploy-only.yml — emergency manual dispatch . . . 40
[OK] Phase 5 verification . . . 41
Phase 6: Notifications . . . 41
6.1 Slack mrkdwn is NOT standard Markdown . . . 42
6.2 Use jq -n --arg for JSON, never sed . . . 42
6.3 Weekly pipeline digest (optional but recommended) . . . 43
[OK] Phase 6 verification . . . 44
Phase 7: Observability . . . 44
7.1 The two health endpoints . . . 44
7.2 The /health endpoint (FastAPI example) . . . 45
7.3 The /api/v1/health endpoint (FastAPI example) . . . 45
7.4 Frontend git_sha in the footer . . . 46
7.5 Sentry (optional but recommended) . . . 46
7.6 UptimeRobot (free, 5 minutes to set up) . . . 47
[OK] Phase 7 verification . . . 48
7.7 Schedule security audits as a recurring discipline *(both tracks)* . . . 48
Phase 8: Documentation . . . 49
8.1 docs/CI-CD.md — the canonical pipeline reference . . . 49
8.2 docs/OPERATIONS-RUNBOOK.md — for oncall . . . 49
8.3 Retrospectives . . . 50
8.4 CHANGELOG.md . . . 50
8.5 In-repo rules . . . 50
[OK] Phase 8 verification . . . 50

Track B — PaaS + BaaS **50**
Track B — Pipeline at a Glance . . . 51
Track B Phase 0: Foundation . . . 52
Track B Phase 1: Local Development . . . 53
B1.1 The build is `vite build`. There is no Dockerfile. . . . 53
B1.2 Version stamping with Vite — read `package.json` directly . . . 53
B1.3 BaaS local dev — Supabase as the example . . . 54
B1.4 Environment variables — `.env.local` and `.env.example` . . . 54
B1.5 Security headers via platform config . . . 55
[OK] Track B Phase 1 verification . . . 55
Track B Phase 2: Basic CI . . . 55
[OK] Track B Phase 2 verification . . . 57
Track B Phase 3: CI Self-Protection . . . 57
Track B Phase 4: Atomic Platform Deploy . . . 58
B4.1 The deploy job . . . 58
B4.2 BaaS migrations must run before the frontend deploys . . . 58
B4.3 Rollback . . . 59
B4.4 What does NOT apply to Track B . . . 59
B4.5 What DOES apply to Track B . . . 59
B4.6 The handler pattern — extract edge function boilerplate before it diverges . . . 60
B4.7 Default to anon, opt in to admin . . . 61
B4.8 Pre-deployment configuration ordering — set the prerequisite before deploying the dependency . . . 61
[OK] Track B Phase 4 verification . . . 62

Track B Phase 5: Automated Release . . . 63
Track B Phase 6: Notifications . . . 63
Track B Phase 7: Observability . . . 63
B7.1 What replaces the backend health endpoint . . . 63
B7.2 Post-deploy verification — the visual QA matrix . . . 64
B7.3 Performance and accessibility as ongoing observability . . . 64
B7.4 Client-side Sentry with ErrorBoundary . . . 65
B7.5 UptimeRobot for the public URL and the BaaS status page . . . 65
B7.6 SPA SEO via meta-tag injection at the edge . . . 65
[OK] Track B Phase 7 verification . . . 66
Track B Phase 8: Documentation . . . 66
[OK] Track B Phase 8 verification . . . 67
Track B summary . . . 67
Two Track B case studies ground this section . . . 67
Appendix A: The Gotcha Hall of Fame . . . 68
1. [ORANGE] GitHub default branch vs. git-flow . . . 68
2. [ORANGE] Docker build context is everything in the build dir . . . 68
3. [ORANGE] `target-branch` in release-please-config.json is silently ignored . . . 68
4. [ORANGE] release-please needs a GitHub Release anchor . . . 68
5. [RED] `GITHUB_TOKEN`-created releases don't trigger downstream workflows . . . 69
6. [YELLOW] Slack `mrkdwn` is not standard Markdown . . . 69
7. [RED] `docker compose --env-file` doesn't reliably propagate to build args . . . 69
8. [RED] `/health` vs `/api/v1/health` . . . 69
9. [RED] Bash self-reexec after git pull . . . 69
10. [ORANGE] Commitlint `subject-case` rule . . . 69
11. [YELLOW] `sed` delimiter injection on user input . . . 70
12. [YELLOW] Squash-merge divergence . . . 70
13. [ORANGE] "Evil merges" . . . 70
14. [YELLOW] Hyphens in wordlists become phrase separators . . . 70
15. [ORANGE] `bash -c` + `export VAR=$(...)` + `set -e` interactions . . . 70
16. [ORANGE] Vite + `import.meta.env.npm_package_version` is unreliable *(Track B)* . . . 70
17. [RED] Publishable vs. service-role keys on BaaS *(Track B)* . . . 71
18. [RED] Default-to-admin in BaaS handler signatures *(Track B)* . . . 71
19. [RED] PostgREST `.or()` is an injection vector with user input *(Track B)* . . . 71
20. [RED] Supabase admin checks must read `app_metadata.role`, not `user_metadata.role` *(Track B)* . . . 72
21. [ORANGE] The Guard That Came Before the Secret *(both tracks, but bites Track B more)* 72
22. [YELLOW] Supabase edge function deployment latency *(Track B)* . . . 73
23. [YELLOW] `deno.lock` v5 vs Supabase CLI version mismatch *(Track B)* . . . 73
24. [ORANGE] Netlify Edge Function sandbox restrictions *(Track B)* . . . 73
25. [YELLOW] Vitest version sensitivity to Node version *(Track B, but applies anywhere Vitest runs)* . . . 73
Appendix B: Ship-Readiness Checklist . . . 74
Foundation . . . 74
Local dev . . . 74
CI . . . 74
Deploy . . . 74
Release . . . 74
Observability . . . 75
Docs . . . 75
Gotchas . . . 75
Track B additions . . . 75
Visual QA matrix template . . . 75
Appendix C: Common Commands Cheat Sheet . . . 76

Development . . . 76
Git . . . 76
Deploy & operations . . . 77
Debugging . . . 77
Closing . . . 78

The DevOps Pipeline Playbook

What this is: a step-by-step guide to building a comprehensive, production-grade CI/CD pipeline on a new project from scratch. It is designed so a college intern with programming chops but no devops background, or a small AI agent with moderate capability, can work through it top-to-bottom and end up with a pipeline that lints itself, tests itself, blue-green deploys, auto-rolls-back, and tells you exactly what's running in production at any moment.

Where the patterns come from: every rule, gotcha, and code snippet in here was earned by shipping a small but real production web application across a couple dozen releases. That project is referenced throughout as a case study; you do not need access to it. Every story this playbook depends on is told in full, right here, and the bugs are named so you can recognize them when you see them in your own pipeline.

How to use this: work the phases in order. Each phase is self-contained — you can stop at any phase and ship what you have. Each phase ends with a **"how you know it worked"** check. Do not skip that check.

Time estimate: 2–4 days of focused work for a new greenfield project. Less if you're adapting an existing repo that already has some of this. More if you get distracted by gotchas (you will; they're marked [WARNING] so you can spot them early).

Two tracks, both first-class: this playbook covers two production-grade deployment shapes, because the modern reader lives in one or the other. **Track A — Self-hosted (VPS + Docker + blue-green)** uses Python/FastAPI + PostgreSQL + Redis on a Linux VPS as the working example. **Track B — Platform-managed (PaaS + BaaS + atomic deploys)** uses React/Vite + Supabase + Netlify as the working example. Both are grounded in real projects shipped at real velocity. Both follow the same six core rules (see Philosophy). The mechanics diverge starting in Phase 1 — Dockerfiles versus platform build configs, deploy scripts versus CLI calls, health endpoints versus visual-QA matrices. Pick your track in "Choose your track" immediately after the Philosophy; you'll work one set of phases end-to-end, with explicit cross-references to the other whenever a principle applies to both.

The patterns in each track transfer. Track A's Python examples apply directly to Node, Go, Ruby, or Rust on any SSH-accessible host. Track B's Netlify/Supabase examples apply directly to Vercel/Cloudflare Pages + Firebase/PlanetScale/Clerk. Where a rule is genuinely stack-specific it's called out. **The playbook does not tell you which track to pick** — both are valid, and the right choice depends on what you're building, what you already have, and what you want to own.

This playbook has a companion

This is one of a pair. You're reading the **DevOps Pipeline Playbook** — how to build the boring machinery: CI that catches itself, deploys that answer for themselves, walls at every boundary. Its companion is the **Engineer + Agent Playbook**, a field manual for working *with* an AI agent inside a pipeline like this one: how to brief it, how to verify its work, how to run the retro that keeps the partnership honest.

They're meant to be read together but each stands alone:

- **Read this one** if you're building infrastructure from scratch and need the pipeline to stop leaking bugs.
- **Read the other** if the pipeline exists but your human-and-agent collaboration is breaking — missed specs, unverified "done," retros that never get written.

Cross-references throughout this document point at specific sections of the Engineer + Agent Playbook where a principle is explored in depth.

Table of Contents

- This playbook has a companion
- Philosophy
- Choose your track
- Pipeline at a Glance
- Prerequisites

Track A — Self-hosted (VPS + Docker + blue-green)

- Phase 0: Foundation
- Phase 1: Local Development
- Phase 2: Basic CI
- Phase 3: CI Self-Protection
- Phase 4: Blue-Green Deploy
- Phase 5: Automated Release
- Phase 6: Notifications
- Phase 7: Observability
- Phase 8: Documentation

Track B — Platform-managed (PaaS + BaaS + atomic deploys)

- Track B Overview
- Track B Phase 0: Foundation
- Track B Phase 1: Local Development
- Track B Phase 2: Basic CI
- Track B Phase 3: CI Self-Protection
- Track B Phase 4: Atomic Platform Deploy
- Track B Phase 5: Automated Release
- Track B Phase 6: Notifications
- Track B Phase 7: Observability
- Track B Phase 8: Documentation

Appendices

- Appendix A: The Gotcha Hall of Fame
- Appendix B: Ship-Readiness Checklist
- Appendix C: Common Commands Cheat Sheet

Philosophy

The six rules at a glance — pin this above your desk.

1. Catch bugs at the nearest wall.
2. Every deploy must be answerable.
3. Fail loudly at known boundaries.
4. The pipeline must also lint itself.

5. Documentation is part of the build.
6. Don't skip the ”boring” work.

The full prose for each rule is below. The rules are ordered by how often they'll save you, not by complexity.

Six rules, learned by being bitten:

1. **Catch bugs at the nearest wall.** A bug caught in CI is 10× cheaper than a bug caught in production. A bug caught on your laptop is 10× cheaper than one caught in CI. Build the linter, the type checker, the smoke test, the manifest validation, the notification — every new wall catches a class of bugs that's otherwise invisible.

2. **Every deploy must be answerable.** At any moment, ”what's in production right now?” must have a precise answer. That means a real git SHA in the health endpoint, a deploy manifest file on the server, a Slack notification trail, and a rollback path that doesn't require you to remember anything.

3. **Fail loudly at known boundaries.** Silent failures are the enemy. If a step can't do its job, it should fail the whole pipeline with a clear error. Defensive retries and graceful degradation are for production behavior; they are NOT for your CI/CD pipeline.

4. **The pipeline must also lint itself.** Workflows that don't parse, shell scripts with injection bugs, commit messages that break automation — these are all bugs in your CI/CD code. Add a linter for them.

5. **Documentation is part of the build.** An operations runbook that points at a deleted script is worse than no runbook. Your docs need to be checked, linked, and updated in the same PRs as the code they describe.

6. **Don't skip the ”boring” work.** The parts that seem like bureaucracy — conventional commits, SHA-pinning actions, writing a retrospective, filing a follow-up Linear issue — are what let the pipeline stay healthy for years instead of months. Every shortcut here is paid back with interest.

   [TIP] **The big insight from the case-study projects' CI/CD work:** every time we added a new ”wall” (a linter, a smoke test, a manifest validator, a Lighthouse budget, an axe-core check), it immediately caught bugs we had NO idea existed. `actionlint` caught 10 pre-existing `SC2086` findings on its first run on the self-hosted project. Lighthouse CI caught accessibility regressions on the platform-managed project before they reached users. The linter isn't overhead — it's a bug detector you're not using.

*For how a human works **with** an agent inside a pipeline like this one — briefing, verifying, retro'ing — see the Engineer + Agent Playbook.*

Choose your track

Before you start Phase 0, decide which track you're working. The Philosophy applies to both. Phase 0 (foundation: git-flow, conventional commits, branch protection, version-as-source-of-truth) applies to both. Phases 5, 6, and 8 are largely shared across tracks. Phases 1, 4, and 7 diverge significantly — different mechanics for the same principles.

This section does not tell you which to pick. It tells you what each is and what it's good for. The right choice depends on your project, your constraints, and what you already have available.

Track A — Self-hosted (VPS + Docker + blue-green)

Working example: Python/FastAPI backend + React frontend + PostgreSQL + Redis on a single Linux VPS, with Docker Compose for orchestration, nginx for the blue-green flip, and a hand-written deploy script

that owns every step. The case study is a small production web app shipped across roughly two dozen releases.

When this fits well:

- You own the hardware, VMs, or want to.
- You need custom server-side logic that doesn't fit serverless constraints — long-lived connections (WebSocket, SSE), large binaries, specialized native dependencies, scheduled jobs, GPU work, anything that wants a process you can SSH into.
- You want full control of the deploy pipeline and the ability to debug it from the inside.
- You have appetite for managing OS updates, nginx config, and SSL certificate rotation — or willingness to learn.
- Your project is enough of a product that "two days to set up the pipeline once" is cheap compared to the lifetime of the system.
- You want the deploy script as artifact: a thing other engineers can read, audit, and modify directly.

Tech-stack assumption: Linux VPS with SSH, Docker + Docker Compose, nginx, GitHub Actions for CI, blue-green script for deploy. Patterns transfer to any SSH-accessible host (DigitalOcean, Linode, Hetzner, AWS EC2, GCP Compute Engine, bare metal) and any backend language. Frontend can be anything; the deploy mechanism is host-side.

Phases: the original Phase 0 through Phase 8 of this playbook are the Track A phases. Work them in order.

Track B — Platform-managed (PaaS + BaaS + atomic deploys)

Working example: React/Vite frontend on Netlify with Supabase as the managed PostgreSQL + Auth + RLS backend, GitHub Actions for CI, Playwright + Lighthouse CI + axe-core as the gates, and the platform handling deploy orchestration and rollback. The case study is a time-tracking web app shipped through nine major versions in three days.

When this fits well:

- Your project is frontend-primary or frontend-only — the backend is mostly schema, auth, and occasional RPC functions.
- You're comfortable with managed services owning the database, auth, and (often) functions layer. RLS is your authorization boundary.
- You're optimizing for "ship fast with a small team" over "control every layer."
- You want the platform to handle blue-green for you — atomic deploys, instant rollback via dashboard, no script to write.
- You're willing to trade some flexibility for less infrastructure to maintain.
- Your verification can be visual + automated (Playwright, Lighthouse, axe-core) rather than curl-against-an-endpoint.

Tech-stack assumption: Vite or Next.js or similar static/serverless frontend, Netlify or Vercel or Cloudflare Pages or similar PaaS host, Supabase or Firebase or PlanetScale or similar BaaS data layer, GitHub Actions for CI, Playwright for E2E, Lighthouse CI for performance and accessibility budgets, client-side Sentry for error monitoring. Patterns transfer across the whole PaaS+BaaS ecosystem; the specific platform CLI commands change but the structure doesn't.

Phases: see Track B — PaaS + BaaS after Phase 8. Each Track B phase explicitly cross-references its Track A counterpart so you can jump between them when a principle is shared.

A short decision aid (not a verdict)

Neither column is a score. If most of one column applies to you, that track probably fits. If you're split down the middle, build a small thing on each and see which feels right for your project.

Question	Track A leans toward...	Track B l
Where does the heavy logic live?	A custom backend you own end-to-end	The schem
What's your ops appetite?	High — you want to own the OS, nginx, SSL, deploy script	Low — yo
What's your team size?	Any, but the pipeline pays off most when several engineers will read it	Small or s
How much custom server work?	Significant — long-lived connections, jobs, GPU, binaries, specialized deps	Minimal -
What does "ship" mean to you?	A reproducible deploy artifact you can audit and roll back by hand	A green C
Cost profile?	Low fixed VPS cost, you scale by adding boxes	Low at lo
What does "verification" feel like?	`curl /api/v1/health` and read the JSON	Open the
Lock-in tolerance?	Low — your stack is portable across hosts that give you SSH	Higher —

The two tracks are not in opposition. A single team might run Track A for one project and Track B for the next. A single project might start on Track B and graduate to Track A when the backend grows enough custom logic to deserve its own server. The playbook covers both because both are real choices that real teams make for real reasons.

Pipeline at a Glance

> *The diagrams below show the **Track A (self-hosted)** pipeline. For the Track B pipeline diagrams (atomic platform deploy + BaaS migrations), see Track B — Pipeline at a Glance at the start of the Track B section.*

Three diagrams. Read them top-to-bottom before starting Phase 0 and you'll have a mental model of where everything is going.

1. The full release chain — commit to production

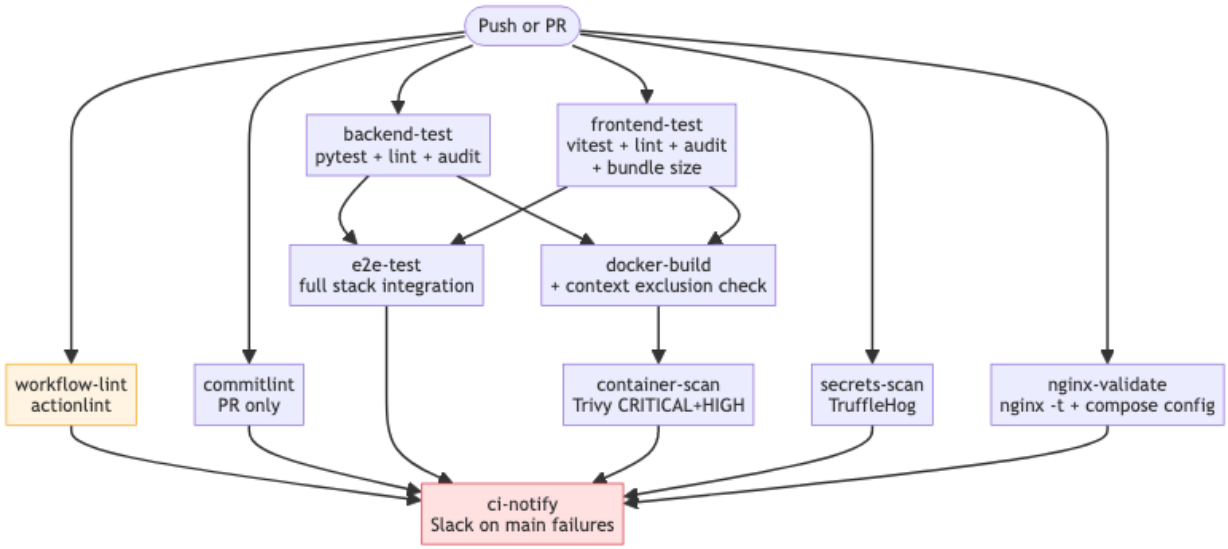
This is what a developer sees from writing code to seeing it live.

Key transitions:

- [BLUE] `Merge to main` — the commit that triggers release automation (Phase 5)
- [RED] `GitHub Release published` — the event that triggers the production deploy. **Critical:** this only fires if release-please uses a PAT, NOT `GITHUB_TOKEN`. See Gotcha Hall of Fame #5.
- [GREEN] `Live site updated` — you can `curl` and verify (Phase 7)

2. CI job graph — what runs on every push and PR

Inside `ci.yml`. Job dependencies are enforced via `needs:` — jobs without dependencies run in parallel.

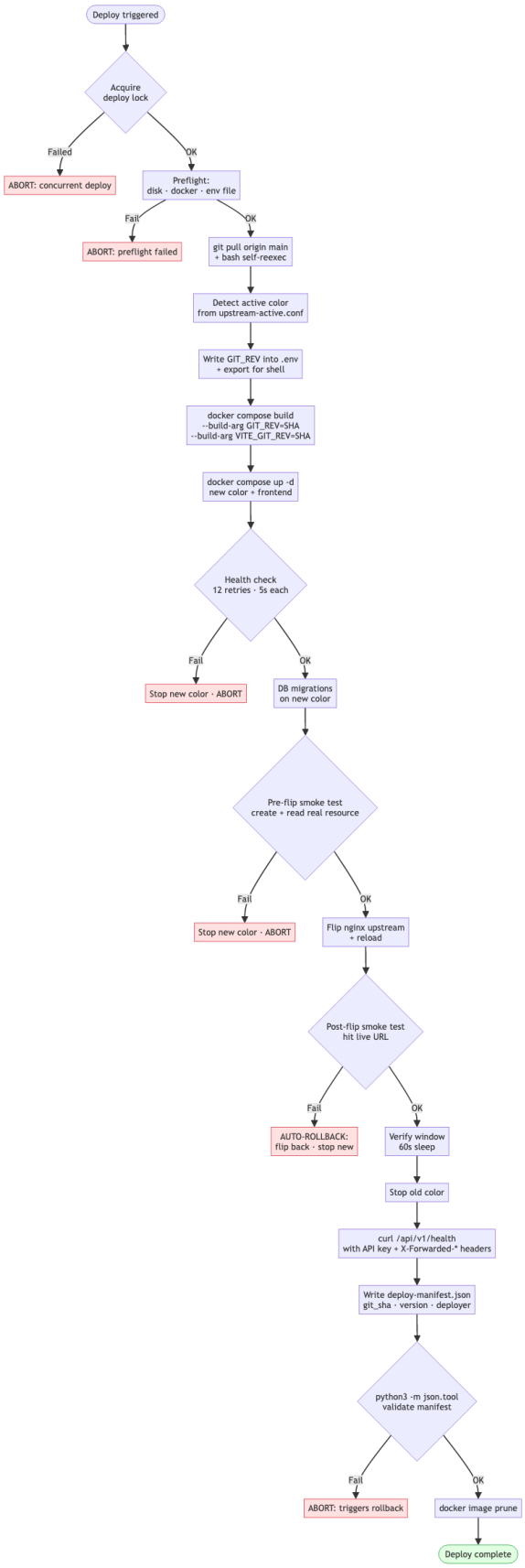


Architecture notes:

- [YELLOW] `workflow-lint` runs FIRST and fastest (~5s). It's cheap and self-protects the pipeline. When it fires on your own repo for the first time, expect it to find real bugs. See Phase 3.
- **Parallel by default, sequential only when needed.** The only `needs:` relationships are (a) `e2e` and `docker-build` need the unit test jobs to pass first so you don't burn 5 minutes on a broken build, (b) `container-scan` needs the Docker image from `docker-build`.
- [RED] `ci-notify` fires ONLY on `main` branch failures, not on PRs. PR failures are expected and don't need to wake anyone up.

3. Blue-green deploy — what happens on the server

What `deploy/deploy-bluegreen.sh` does when SSH'd into by the deploy job. Every decision point has a named abort or rollback path.


Deploy triggered
Acquire deploy lock
Failed
OK
ABORT: concurrent deploy
Preflight: disk · docker · env file
Fail
OK
ABORT: preflight failed
git pull origin main + bash self-reexec
Detect active color from upstream-active.conf
Write GIT_REV into .env + export for shell
docker compose build --build-arg GIT_REV=SHA --build-arg VITE_GIT_REV=SHA
docker compose up -d new color + frontend
Health check 12 retries · 5s each
Fail
OK
Stop new color · ABORT
DB migrations on new color
Pre-flip smoke test create + read real resource
Fail
OK
Stop new color · ABORT
Flip nginx upstream + reload
Post-flip smoke test hit live URL
Fail
OK
AUTO-ROLLBACK: flip back · stop new
Verify window 60s sleep
Stop old color
curl /api/v1/health with API key + X-Forwarded-* headers
Write deploy-manifest.json git_sha · version · deployer
python3 -m json.tool validate manifest
Fail
OK
ABORT: triggers rollback
docker image prune
Deploy complete

Read the diagram as a decision tree:

- **Before the flip** (health check, pre-flip smoke), ANY failure is safe: the old version is still serving, the new version gets stopped, the deploy exits non-zero. Nothing is broken.
- **After the flip** (post-flip smoke), failure triggers **automatic rollback**: flip nginx back to the old color, stop the new color, exit non-zero. The failure window is the few seconds between the flip and the post-flip smoke — measured in seconds, not minutes.
- **After the verify window**, a manifest validation failure still triggers the existing rollback path. The cheapest safety net is the last one.

Critical patterns you'll see referenced throughout the playbook:

- **Self-reexec after git pull** (Gotcha Hall of Fame #9) — bash buffers scripts in memory
- **`--build-arg` on `docker compose build`** (Gotcha Hall of Fame #7) — compose YAML substitution is unreliable for build args
- **`/api/v1/health` not `/health`** for verification (Gotcha Hall of Fame #8) — the two endpoints return different things
- **`python3 -m json.tool` on the manifest** — cheap insurance against shell quoting corruption

---

Prerequisites

Before you start, you need:

- ☐ A GitHub repository (empty is fine)
- ☐ A cloud host (AWS, DigitalOcean, Hetzner, Linode, Fly.io — anything that gives you SSH access)
- ☐ A domain name pointed at your host
- ☐ Basic familiarity with: git, bash, Docker, GitHub Actions YAML. Don't panic if any of these are new — the playbook explains them as it goes.
- ☐ A project you want to deploy. It should have a backend (Python/Node/Go/whatever) and optionally a frontend. This playbook uses Python FastAPI + React for examples, but the patterns transfer directly.

Tools to install locally:

```
# macOS (Homebrew)
brew install git gh docker docker-compose jq python3 node

# Also install:
# - Docker Desktop (for docker + docker compose)
# - gh CLI authenticated: gh auth login
```

Secrets you'll need access to (gather these early, you'll need them at various points):

- A Slack webhook URL (for notifications) — create via Slack → Apps → Incoming Webhooks
- SSH private key that can log into your production host
- A GitHub Personal Access Token with `contents: write` and `pull-requests: write` scope (for release automation)
- (Optional) Sentry DSN, Prometheus Pushgateway URL, anything else for observability

---

Track A — Self-hosted (VPS + Docker + blue-green)

> *The phases below build the self-hosted track end-to-end. If you're on the platform-managed track instead (Vite + Supabase + Netlify or similar), jump to Track B — PaaS + BaaS after this Phase 0; Phase 0 is shared between both tracks. Each Track B phase explicitly cross-references its Track A counterpart, so you can hop between them when a principle is shared.*

Phase 0: Foundation (Before Anything Else)

> *Shared between both tracks.* Git-flow, conventional commits, branch protection, version-as-source-of-truth, and `.gitignore`/`.dockerignore` hygiene apply equally to Track A and Track B. The only Track B delta is that the version source of truth is `package.json` rather than a separate `VERSION` file. Track B readers can work this entire phase, then jump to Track B Phase 1: Local Development.

Goal: set up the scaffolding that every later phase depends on. Skip any of this and you'll be ripping things out later.

Time: ~1 hour

0.1 Git-flow branching

This playbook uses **two protected branches**:

- `develop` — integration branch, default branch, where all feature PRs land
- `main` — release branch, only develop→main merges go here, triggers releases and deploys

```
cd your-project
git init
git checkout -b develop
git commit --allow-empty -m "chore: initial commit"
git remote add origin git@github.com:YOUR-ORG/YOUR-REPO.git
git push -u origin develop

# Create main from develop
git checkout -b main
git push -u origin main

# Go back to develop for normal work
git checkout develop
```

Then in GitHub UI:

- **Settings → Branches → Default branch → change to `develop`**
- **Settings → Branches → Add rule for `main`:**
  - Require PR before merging
  - Require status checks to pass (add them later when CI exists)
  - Include administrators — check it; it keeps honest mistakes from landing

> [WARNING] **Gotcha:** if you don't change the default branch to `develop` NOW, release-please (phase 5) will silently target the wrong branch and produce wildly wrong releases. We lived this. See Appendix A entry #1.

0.2 Conventional commits + commitlint

Every commit message you write from now on follows this format:

```
type(scope): short description

optional longer body explaining why
```

Where `type` is one of: `feat`, `fix`, `docs`, `style`, `refactor`, `perf`, `test`, `chore`, `build`, `ci`, `revert`.

Why: this lets release-please generate version bumps and CHANGELOGs automatically from your commit history. It's also how `commitlint` can enforce "no more 'wip' or 'stuff' commits on main." *(The Engineer + Agent Playbook §2 argues this class of hygiene is the precondition for the workspace, not an extra — read it if you need to convince a teammate.)*

Add commitlint to the repo:

```
# At repo root
npm init -y
npm install --save-dev @commitlint/cli @commitlint/config-conventional

cat > commitlint.config.js <<'EOF'
module.exports = {
  extends: ['@commitlint/config-conventional']
};
EOF

git add package.json package-lock.json commitlint.config.js
git commit -m "chore: add commitlint config"
```

> [WARNING] **Gotcha:** commitlint's default `subject-case` rule disallows sentence-case/start-case/pascal-case/upper-case on the subject. Your subjects must start with a lowercase letter. `fix(ci): Target main branch` will FAIL commitlint. `fix(ci): target main branch` will pass. Watch for this when pasting issue tracker IDs — don't put them at the start of the subject. Put them in parens at the end instead.

0.3 .gitignore and .dockerignore

`.gitignore`: standard, language-specific. Hit up gitignore.io for a good starting point.

`.dockerignore`: THIS IS CRITICAL and almost always wrong by default. Your Docker build context is everything in the repo root unless excluded. A bad `.dockerignore` means your build uploads the `.git` folder, `node_modules`, `__pycache__`, screenshots, and test databases to Docker every single time, making builds 10× slower and creating a security risk (your git history leaks into the image).

Start with this aggressive baseline:

```
# .dockerignore - at repo root
.git
.github
.gitignore
.dockerignore

# Python
__pycache__
*.pyc
*.pyo
*.pyd
.pytest_cache
.mypy_cache
.ruff_cache
.venv
venv
*.egg-info

# Node
node_modules
.npm
dist
build
coverage
```

```
# Editors / OS
.vscode
.idea
.DS_Store
*.swp
*.swo

# Project-specific
logs
*.log
.env
.env.*
!.env.example
backups
tmp
```

Verify it's working:

```
# Show the actual build context size
docker build --no-cache -f backend/Dockerfile -t test-build . 2>&1 | head -5
# Look for "Sending build context to Docker daemon  X MB" - if it's over ~50 MB for a small project, yo
```

> [TIP] **From the case-study project:** a `.dockerignore` rewrite dropped the build context from 50+ MB to 11.7 MB. The big wins were excluding `.git/` (which always grows), a stray `monitoring/` directory, and a folder of screenshots left over from a marketing sprint that no one had thought to gitignore. Audit yours after a few months — directories sneak in.

0.4 The VERSION file

Create a file called `VERSION` at the repo root containing your current version:

```
echo "0.1.0" > VERSION
git add VERSION
git commit -m "chore: add VERSION file"
```

Why: this becomes the single source of truth for your project version. Your backend, frontend, Dockerfile, CI, and release automation all read from this file. When you bump the version, you bump it here and everything else flows.

Future phases will make release-please auto-bump this file.

0.5 First commit to main

You need main to have at least one commit that tracks develop so later merges work:

```
git checkout main
git merge develop --ff-only
git push origin main
git checkout develop
```

[OK] Phase 0 verification

- ☐ `git branch -a` shows both `develop` and `main` (local and origin)
- ☐ GitHub → Settings → Branches shows `develop` as default and `main` as protected
- ☐ `cat VERSION` returns `0.1.0`
- ☐ `cat .gitignore` and `cat .dockerignore` exist and look sensible
- ☐ `npx commitlint --from HEAD~1 --to HEAD` passes (or gracefully no-ops if there's only one commit)

- ☐ You can make a commit with a bad message and commitlint catches it: `git commit --allow-empty -m "Bad Message With Caps"` → fails

---

Phase 1: Local Development

Goal: everything you need to run the project locally via `docker compose up`.

Time: 2–4 hours depending on project complexity

1.1 Multi-stage Dockerfiles

Every service in your project gets its own Dockerfile. Use **multi-stage builds** so your production image doesn't contain build tools, test dependencies, or `.git`.

Example: Python backend (`backend/Dockerfile`)

```
# Stage 1: dependencies
FROM python:3.11-alpine AS deps
WORKDIR /app
RUN apk add --no-cache gcc musl-dev postgresql-dev libffi-dev
COPY backend/requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Stage 2: test (used by CI, not by prod)
FROM deps AS test
COPY backend/ .
CMD ["python", "-m", "pytest", "tests/", "-v"]

# Stage 3: production (what ships)
FROM python:3.11-alpine AS production
WORKDIR /app
RUN apk add --no-cache postgresql-client
COPY --from=deps /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages
COPY --from=deps /usr/local/bin /usr/local/bin
COPY backend/ .
COPY VERSION /app/VERSION

# Accept build args so CI can stamp the image with git metadata
ARG GIT_REV=unknown
ENV GIT_REV=$GIT_REV

# Run as non-root
RUN adduser -D -s /bin/sh appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Key patterns to copy:

1. **Separate stages for deps / test / production.** CI builds the `test` stage, production builds the `production` stage.
2. **`ARG GIT_REV=unknown`** at the production stage. Later phases pass the real SHA via `--build-arg`.
3. **`ENV GIT_REV=$GIT_REV`** — promotes the build arg to a runtime env var that your app can read.

4. **Copy VERSION into the image.** Your app can read it at startup for display.
5. **Non-root user.** Alpine's `adduser -D` is the minimal form.

   [WARNING] **Gotcha:** `ARG` lines invalidate the Docker layer cache from that point down. Put them as late as possible in the Dockerfile so cached dependency layers aren't rebuilt every time the SHA changes.

1.2 docker-compose.yml for local dev

```yaml
# docker-compose.yml - local development
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: dev_user
      POSTGRES_PASSWORD: dev_password
      POSTGRES_DB: myapp
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass dev_redis_pw
    ports:
      - "127.0.0.1:6379:6379"

  backend:
    build:
      context: .
      dockerfile: backend/Dockerfile
      target: production
      args:
        GIT_REV: ${GIT_REV:-local-dev}
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      DATABASE_URL: postgresql://dev_user:dev_password@db:5432/myapp
      REDIS_URL: redis://:dev_redis_pw@redis:6379/0
      GIT_REV: ${GIT_REV:-local-dev}
    depends_on:
      - db
      - redis

  frontend:
    build:
      context: .
      dockerfile: frontend/Dockerfile
      target: production
      args:
        VITE_API_URL: ${VITE_API_URL:-http://localhost:8000}
        VITE_GIT_REV: ${GIT_REV:-local-dev}
    ports:
      - "127.0.0.1:3000:3000"
```

```yaml
    depends_on:
      - backend

volumes:
  db_data:
```

Pattern notes:

- **Port binding `127.0.0.1:8000:8000`** — NOT `0.0.0.0:8000`. This prevents your dev services from being exposed on your local network.
- **`target: production`** — use the same image locally as you ship to prod. "It works on my machine with a dev server, but fails in prod" is a category of bugs you can eliminate entirely.
- **Build args inline, with defaults.** `${GIT_REV:-local-dev}` means "use GIT_REV from env if set, else `local-dev`." Works the same in CI and locally.

1.3 Production compose file

Create a separate compose file for prod deployment. This is the one the deploy script uses on the server.

```yaml
# docker-compose.prod.yml
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - /var/lib/myapp/db:/var/lib/postgresql/data
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
    volumes:
      - /var/lib/myapp/redis:/data
    restart: unless-stopped

  # Blue-green: two backend services, only one routed at a time
  backend-blue:
    build:
      context: .
      dockerfile: backend/Dockerfile
      target: production
      args:
        GIT_REV: ${GIT_REV:-unknown}
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379/0
      GIT_REV: ${GIT_REV:-unknown}
    depends_on:
      - db
      - redis
    restart: unless-stopped
```

```yaml
  backend-green:
    build:
      context: .
      dockerfile: backend/Dockerfile
      target: production
      args:
        GIT_REV: ${GIT_REV:-unknown}
    ports:
      - "127.0.0.1:8001:8000"
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379/0
      GIT_REV: ${GIT_REV:-unknown}
    depends_on:
      - db
      - redis
    restart: unless-stopped

  frontend:
    build:
      context: .
      dockerfile: frontend/Dockerfile
      target: production
      args:
        VITE_API_URL: ${BACKEND_URL}
        VITE_GIT_REV: ${GIT_REV:-unknown}
    ports:
      - "127.0.0.1:3000:3000"
    restart: unless-stopped
```

[WARNING] **Critical:** `GIT_REV=${GIT_REV:-unknown}` in the `environment:` section DOES propagate at runtime IF the shell (or env file) has GIT_REV set. But the `args: GIT_REV: ${GIT_REV:-unknown}` in the `build:` section may NOT propagate reliably at build time via compose's YAML substitution — the case-study project hit this exact bug and lived in it for three deploys before catching it (see *The Health Check That Wasn't*, Appendix A entry #8). **The reliable fix is to pass `--build-arg GIT_REV=...` explicitly on the `docker compose build` command line.** Phase 4 shows how.

1.4 .env.example

```bash
# .env.example - commit this. Users copy to .env and fill in real values.
POSTGRES_USER=myapp
POSTGRES_PASSWORD=CHANGE_ME
POSTGRES_DB=myapp
REDIS_PASSWORD=CHANGE_ME
BACKEND_URL=https://your-domain.com
SECRET_KEY=CHANGE_ME_32_chars_minimum_for_real_use
```

Add `.env` (without the `.example`) to `.gitignore` — real secrets must NEVER be committed.

[OK] Phase 1 verification

```bash
cp .env.example .env
# Fill in .env with local dev values (dev_password is fine for local)
docker compose up -d --build
```

```
sleep 10  # let services start

# Every service should be healthy:
docker compose ps

# Frontend responds:
curl -s http://localhost:3000 | head -5

# Backend responds:
curl -s http://localhost:8000/health  # you may need to add this endpoint

# Shut down
docker compose down
```

Expected: all services "running", curl returns HTTP content (not connection refused), and `docker compose down` cleans up without errors.

Phase 2: Basic CI

Goal: every push and PR runs automated tests against your backend and frontend. Broken code never gets to main.

Time: 2–3 hours

2.1 The skeleton

Create `.github/workflows/ci.yml`:

```yaml
name: CI

on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop, main]
  workflow_call:  # lets other workflows (deploy.yml) reuse this

permissions:
  contents: read

jobs:
  # Jobs will be filled in below
```

`workflow_call` lets Phase 4's deploy workflow re-run CI before deploying. Don't omit it.

2.2 Commit Message Lint

```yaml
  commitlint:
    name: Commit Message Lint
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'

    steps:
      - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1
        with:
```

```yaml
      fetch-depth: 0

  - name: Set up Node.js 18
    uses: actions/setup-node@49933ea5288caeca8642d1e84afbd3f7d6820020 # v4.4.0
    with:
      node-version: "18"
      cache: npm

  - run: npm ci
  - run: npx commitlint --from ${{ github.event.pull_request.base.sha }} --to ${{ github.event.pull_
```

[TIP] **Why SHA-pin actions?** `actions/checkout@v4` can change what it does when v4.3.2 is released. A malicious actor who compromises the checkout action can now run arbitrary code in your pipeline. Pinning to a specific commit SHA means you know EXACTLY what code runs. The comment `# v4.3.1` is just for human readability. Every `uses:` in this playbook is SHA-pinned. See Phase 3 for the linter that catches unpinned actions.

2.3 Backend tests

```yaml
backend-test:
  name: Backend Tests
  runs-on: ubuntu-latest

  services:
    postgres:
      image: postgres:15-alpine
      env:
        POSTGRES_USER: test_user
        POSTGRES_PASSWORD: test_password
        POSTGRES_DB: test_myapp
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

    redis:
      image: redis:7-alpine
      ports:
        - 6379:6379
      options: >-
        --health-cmd "redis-cli ping"
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

  steps:
    - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1

    - name: Set up Python 3.11
      uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5.6.0
      with:
        python-version: "3.11"
```

```yaml
          cache: pip
          cache-dependency-path: backend/requirements.txt

      - name: Install dependencies
        run: |
          cd backend
          pip install --upgrade pip
          pip install -r requirements.txt
          pip install ruff pip-audit

      - name: Audit dependencies for vulnerabilities
        run: |
          cd backend
          pip-audit -r requirements.txt

      - name: Lint with ruff
        run: |
          cd backend
          ruff check app/ --select E,F,W --ignore E501

      - name: Run tests
        env:
          DATABASE_URL: postgresql://test_user:test_password@localhost:5432/test_myapp
          REDIS_URL: redis://localhost:6379/0
          SECRET_KEY: ci-test-secret-key-that-is-at-least-32-chars-long
        run: |
          cd backend
          python -m pytest tests/ -v --tb=short --cov=app --cov-fail-under=70
```

Pattern notes:

- **Real PostgreSQL + Redis as CI services, not mocks.** Tests that pass against fakes and fail against the real thing aren't tests.
- **`pip-audit` catches CVEs on every run.** The case-study project surfaced a real `requests` CVE and a critical axios SSRF within hours of first adding the check — both had been in the dependency tree, unflagged, for weeks.
- **Coverage floor (`--cov-fail-under=70`)** prevents silent coverage erosion.

2.4 Frontend tests

```yaml
  frontend-test:
    name: Frontend Tests
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1

      - name: Set up Node.js 18
        uses: actions/setup-node@49933ea5288caeca8642d1e84afbd3f7d6820020 # v4.4.0
        with:
          node-version: "18"
          cache: npm
          cache-dependency-path: frontend/package-lock.json

      - run: cd frontend && npm ci
```

```yaml
      - name: Audit dependencies
        run: |
          cd frontend
          npm audit --audit-level=critical --omit=dev

      - name: Lint
        run: cd frontend && npm run lint

      - name: Run tests
        run: cd frontend && npm run test:ci

      - name: Build and check bundle size
        run: |
          cd frontend
          npm run build
          npx size-limit
```

Add a `.size-limit.json` to frontend/ to set your bundle budget:

```json
[
  {
    "path": "frontend/dist/assets/*.js",
    "limit": "250 kb",
    "gzip": true
  }
]
```

> [TIP] **Why the bundle budget?** Frontend bundles only grow. Without a budget, you wake up one day with a 2 MB payload and no idea when it happened. The budget makes every +10 KB PR a visible number you have to justify.

2.4a Performance and accessibility budgets — Lighthouse CI

> *Applies to both tracks.* Treat the budgets as hard constraints.

Lighthouse CI runs Google Lighthouse on every push and asserts numeric thresholds. Performance, accessibility, and best-practices regressions all fail CI before a reviewer sees them — bundle-size budgets applied to a different class of bug.

Add `lighthouserc.cjs` at the project root (or frontend root):

```cpp
module.exports = {
  ci: {
    collect: {
      staticDistDir: './dist',          // or './frontend/dist'
      isSinglePageApplication: true,
      url: ['http://localhost/'],       // and any other key routes
      numberOfRuns: 3,                  // average across runs to reduce noise
    },
    assert: {
      assertions: {
        'categories:performance':    ['warn', { minScore: 0.9 }],
        'categories:accessibility':  ['warn', { minScore: 0.9 }],
        'first-contentful-paint':    ['warn', { maxNumericValue: 1500 }],
        'interactive':               ['warn', { maxNumericValue: 2500 }],
        'largest-contentful-paint':  ['warn', { maxNumericValue: 2500 }],
```

```
      },
    },
    upload: {
      target: 'temporary-public-storage',  // or your own LHCI server
    },
  },
}
```

Add the CI job:

```yaml
  lighthouse:
    name: Lighthouse CI
    runs-on: ubuntu-latest
    needs: frontend-test    # build artifact must exist

    steps:
      - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1
      - uses: actions/setup-node@49933ea5288caeca8642d1e84afbd3f7d6820020 # v4.4.0
        with:
          node-version: 20
          cache: npm

      - run: npm ci
      - run: npm run build

      - name: Run Lighthouse CI
        run: npx @lhci/cli autorun
        env:
          LHCI_GITHUB_APP_TOKEN: ${{ secrets.LHCI_GITHUB_APP_TOKEN || '' }}

      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}
        with:
          name: lighthouse-report
          path: .lighthouseci/
          retention-days: 14
```

[TIP] **Start with `warn`, not `error`, for the first week.** Lighthouse scores are noisy on a fresh project. Run for a few days, observe what your real numbers look like, then promote to `error` once you trust the floor. Promoting too early teaches the team to ignore Lighthouse failures. *Pin: budgets that fail noisily on day one get muted; budgets that warn on day one and fail in week two get respected.*

[WARNING] **Lighthouse runs need a built static directory.** `staticDistDir` points at the production build. If your build emits to a non-default path (or only with environment variables set), the Lighthouse job will collect zero data and assert against nothing — a green check that means nothing. Always test the Lighthouse run locally first by building, then `npx @lhci/cli autorun`, and confirm the report contains the URLs you expected.

2.4b Accessibility tests — axe-core in E2E

Applies to both tracks. A11y regressions are a recurring bug class on every frontend project, and they're catchable at CI time with one library and a Playwright test.

Install `@axe-core/playwright` and add an accessibility check to your E2E suite:

```
// e2e/a11y.spec.ts
```

```
import { test, expect } from '@playwright/test'
import AxeBuilder from '@axe-core/playwright'

const PAGES = ['/', '/login', '/dashboard', '/settings']

for (const path of PAGES) {
  test(`a11y: ${path}`, async ({ page }) => {
    await page.goto(path)
    const results = await new AxeBuilder({ page })
      .withTags(['wcag2a', 'wcag2aa'])
      .analyze()
    expect(results.violations).toEqual([])
  })
}
```

Run it inside the existing E2E job or as its own. axe-core failures fail CI the same as a unit test failure.

> [TIP] **The first run will find things.** Like `actionlint`, axe-core surfaces real WCAG violations on any existing codebase. Don't disable the test — fix the violations. Most are one-line (missing `aria-label`, contrast issue, button-that-should-be-a-link). The case-study Track B project ran axe-core for the first time in v6.0 and the findings became that release's theme.

2.4c E2E runs on at least one touch device profile

> *Applies to both tracks.* Desktop-only E2E is a blind spot for any project with mobile users.

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  testDir: './e2e',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  projects: [
    { name: 'chromium',     use: { ...devices['Desktop Chrome'] } },
    { name: 'mobile-chrome', use: { ...devices['Pixel 5'] } },
  ],
  // ...
})
```

Touch has its own bug class — `pointerdown` vs `touchstart`, drag-to-scroll conflicts, hover-only affordances that vanish on touch, tap targets too small for fingers. Desktop E2E catches none of them. One mobile profile catches most of them for ~30 extra seconds of CI.

> **Field note — from the Track B case study:** *The iPad Tap That Wasn't.* A frontend shipped a foundation-layer migration with a green E2E suite. Within 24 hours, a user reported tapping a cell on iPad did nothing. Root cause: a touch-vs-mouse guard written and tested on desktop only that conflated tap with drag. Fix was four lines. The test suite never exercised the actual gesture on the actual device. A single `page.tap()` on a `Pixel 5` profile would have caught it in five seconds. Two of that project's three release-day hotfixes traced back to the same gap. Narrow read: add a mobile profile to your Playwright config. Wider read: **migration testing must cover interaction paths, not just data paths.** Run the actual gesture on the actual device through the actual new substrate, or your green check is answering a question you weren't asked.

2.4d Rate-limit every public endpoint, not just the authenticated ones

> *Applies to both tracks.* Public is not the same as unprotected.

Auth gives you per-user rate limiting for free. Unauthenticated endpoints have no caller identity, so they're easy to forget — and they're the easiest endpoints to DoS, because anyone on the internet can hit them at full speed.

Rule: rate limit every endpoint, full stop. IP-based (hashed, GDPR-friendly) for unauthenticated, user-based for authenticated, the more restrictive of the two when both apply. Don't carve exemptions — "public read endpoints" are exactly what gets scraped and weaponized.

> **Field note — from a Track B case study:** *Four Endpoints Without a Throttle.* A web app shipped four unauthenticated public endpoints — categories, graph-data, leaderboard, quiz scores — without rate limits. The limit middleware was wired into the *authenticated* handler base; the four public endpoints used a different handler nobody had wrapped. A v9.3 security audit caught it. The deeper lesson: **the limit middleware should default to "limited" and require explicit opt-out.** The version where you remember to add the limit every time is the version where you eventually forget. Make the safe choice the default.

2.4e Body size limits on POST and PUT

> *Applies to both tracks.* Reject oversized payloads early, before parsing.

Without a cap, one attacker can exhaust your server's memory with a multi-gigabyte JSON POST. The fix is cheap: respond `413 Payload Too Large` when `Content-Length` exceeds your cap, before the body hits the parser.

64KB is a generous cap for application-level POSTs. File upload endpoints need their own higher limit, applied per-endpoint. Wire it at the framework level (FastAPI middleware, Express middleware, edge function handler wrapper) — same default-to-limited logic as the rate-limit rule above.

2.5 Secrets scanning

```yaml
secrets-scan:
  name: Secrets Scanning
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1
      with:
        fetch-depth: 0

    - name: Run TruffleHog
      uses: trufflesecurity/trufflehog@47e7b7cd74f578e1e3145d48f669f22fd1330ca6 # v3.94.3
      with:
        extra_args: --only-verified
```

TruffleHog scans git history for credentials and only reports findings it can **verify are actually valid** (it tries to authenticate with them). `--only-verified` keeps the noise down.

[OK] Phase 2 verification

Push your branch and open a PR. You should see:

- `Commit Message Lint` — pass if your last commit follows conventional format
- `Backend Tests` — pass if you have at least one test file
- `Frontend Tests` — pass if you have at least one test
- `Secrets Scanning` — pass (no findings unless you've actually committed a secret)

Then deliberately break something and confirm CI catches it:

- Commit a message like `This Is Bad` — commitlint should fail
- Add `SECRET_KEY="sk-real-looking-key"` to a file — secrets-scan should catch it
- Remove a test that covers important code — coverage check should fail

---

Phase 3: CI Self-Protection

Goal: the pipeline audits itself. A broken workflow file should be caught by CI, not by the universe.

Time: 1–2 hours

3.1 Workflow Lint (actionlint) — FIRST JOB

Add this to `ci.yml` **as the first job**:

```yaml
  workflow-lint:
    name: Workflow Lint (actionlint)
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1

      - name: Install actionlint
        run: |
          ACTIONLINT_VERSION=1.7.7
          curl -sSL "https://github.com/rhysd/actionlint/releases/download/v${ACTIONLINT_VERSION}/actio
            | tar -xz -C /tmp actionlint
          sudo mv /tmp/actionlint /usr/local/bin/actionlint
          actionlint -version

      - name: Run actionlint
        run: actionlint -color
```

What it catches: workflow schema errors, expression syntax bugs (exactly the class that silently broke an emergency-deploy workflow on the case-study project for months — nobody noticed until an incident), shellcheck findings on inline `run:` blocks (SC2086 etc.), invalid context refs, unknown actions, duplicate job IDs.

> [WARNING] **The case-study project's first actionlint run found 10 pre-existing `SC2086` findings** in `ci.yml` and `release.yml` — unquoted `$GITHUB_ENV` / `$GITHUB_STEP_SUMMARY` / `$GITHUB_OUTPUT` redirects that had been sitting there for months. **Expect your first run to find bugs you didn't know existed.** That's the point.

3.2 Fix the findings

Fix SC2086 issues by quoting:

```yaml
# Wrong:
run: echo "JOB_START=$(date +%s)" >> $GITHUB_ENV

# Right:
run: echo "JOB_START=$(date +%s)" >> "$GITHUB_ENV"
```

Apply the same fix to any unquoted `$GITHUB_*` variable: `$GITHUB_OUTPUT`, `$GITHUB_STEP_SUMMARY`, etc.

3.3 Docker Build Validation

```yaml
docker-build:
  name: Docker Build Validation
  runs-on: ubuntu-latest
  needs: [backend-test, frontend-test]

  steps:
    - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@4d04d5d9486b7bd6fa91e7baf45bbb4f8b9deedd # v4.0.0

    - name: Build backend image
      uses: docker/build-push-action@bcafcacb16a39f128d818304e6c9c0c18556b85f # v7.1.0
      with:
        context: .
        file: backend/Dockerfile
        target: production
        tags: myapp-backend:ci
        load: true
        cache-from: type=gha,scope=backend
        cache-to: type=gha,mode=max,scope=backend

    - name: Verify build context exclusions
      run: |
        VIOLATIONS=""
        for dir in .git node_modules __pycache__ .pytest_cache .venv; do
          if docker run --rm myapp-backend:ci sh -c "test -d /app/$dir" 2>/dev/null; then
            VIOLATIONS="$VIOLATIONS $dir"
          fi
        done
        if [ -n "$VIOLATIONS" ]; then
          echo "::error::Build context exclusion regression - found:$VIOLATIONS"
          exit 1
        fi
        echo "Build context exclusions verified."
```

The `Verify build context exclusions` step is the ratchet. It launches the built image and asserts known-bad directories aren't present. Once your `.dockerignore` is clean, it can't silently regress.

3.4 Container Vulnerability Scan

```yaml
container-scan:
  name: Container Vulnerability Scan
  runs-on: ubuntu-latest
  needs: [docker-build]

  steps:
    - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@4d04d5d9486b7bd6fa91e7baf45bbb4f8b9deedd # v4.0.0

    - name: Build backend image (cached)
```

```yaml
      uses: docker/build-push-action@bcafcacb16a39f128d818304e6c9c0c18556b85f # v7.1.0
      with:
        context: .
        file: backend/Dockerfile
        target: production
        tags: myapp-backend:scan
        load: true
        cache-from: type=gha,scope=backend

    - name: Run Trivy
      uses: aquasecurity/trivy-action@57a97c7e7821a5776cebc9bb87c984fa69cba8f1 # v0.35.0
      with:
        image-ref: myapp-backend:scan
        severity: CRITICAL,HIGH
        exit-code: 1
```

`cache-from: type=gha,scope=backend` reuses the image `docker-build` just built. Saves 2–3 minutes per run.

3.5 CI Failure Notification (main-only)

```yaml
  ci-notify:
    name: CI Failure Notification
    runs-on: ubuntu-latest
    needs: [backend-test, frontend-test, docker-build, container-scan, secrets-scan, workflow-lint]
    if: failure() && github.ref == 'refs/heads/main'

    steps:
      - name: Send Slack CI failure notification
        if: env.DEPLOY_WEBHOOK_URL != ''
        env:
          DEPLOY_WEBHOOK_URL: ${{ secrets.DEPLOY_WEBHOOK_URL }}
        run: |
          PAYLOAD=$(jq -n \
            --arg sha "${{ github.sha }}" \
            --arg url "${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id
            '{
              blocks: [
                {
                  type: "header",
                  text: {type: "plain_text", text: ":rotating_light: CI Failed on main"}
                },
                {
                  type: "section",
                  fields: [
                    {type: "mrkdwn", text: ("*Commit:*\n`" + $sha + "`")},
                    {type: "mrkdwn", text: ("*Action:*\n<" + $url + "|View logs>")}
                  ]
                }
              ]
            }')
          curl -sf -X POST "$DEPLOY_WEBHOOK_URL" \
            -H "Content-Type: application/json" \
            -d "$PAYLOAD" || echo "::warning::Slack notification failed (non-fatal)"
```

Key patterns (full treatment in Phase 6): `jq -n --arg` for JSON construction (not sed — avoids injection), `|| echo "::warning::..."` so a Slack outage doesn't fail the job, fires only on `main` failures (not PRs — PR failures are expected).

[OK] Phase 3 verification

- ☐ Create a new workflow file with a deliberate bug (`run: echo "hi ${{ github.sha }}"` as a single-line quoted scalar) — actionlint should catch it
- ☐ Add a file to your backend source, rebuild — docker-build's context-exclusion check should still pass
- ☐ Temporarily add a known-vulnerable dependency — pip-audit or Trivy should fail the build
- ☐ Push a deliberate CI failure to main (or simulate it) — Slack should get a red notification

---

Phase 4: Blue-Green Deploy

Goal: a deploy script that can ship to production with zero downtime, automatic rollback on failure, and a manifest trail you can audit.

Time: 1 day, including bugs

4.1 The shape of the deploy

Blue-green means you run TWO copies of the backend (blue on port 8000, green on port 8001). At any moment, nginx routes traffic to one of them — that's the "active" color. When you deploy, you:

1. Build the new image
2. Start it as the "new" color (the non-active one)
3. Health-check it, run smoke tests
4. Flip nginx to route to the new color
5. Run more smoke tests after the flip
6. Stop the old color (after a verification window in case you need to roll back)

If anything fails before the flip, you're fine — the old version is still serving. If anything fails after the flip, you flip back (automatic rollback).

4.2 The deploy script

Create `deploy/deploy-bluegreen.sh`. This is long — it's the heart of everything. Read it once to understand the shape, then copy it verbatim and adapt the few things marked `# CUSTOMIZE`.

```bash
#!/usr/bin/env bash
# Blue-green deployment script.
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
PROJECT_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
LOG_DIR="${PROJECT_DIR}/logs"
mkdir -p "$LOG_DIR"
LOG_FILE="${LOG_DIR}/deploy-bluegreen-${TIMESTAMP}.log"
COMPOSE_FILE="docker-compose.prod.yml"  # CUSTOMIZE: your prod compose file name
VERIFY_SECONDS="${BLUEGREEN_VERIFY_SECONDS:-300}"

BLUE_PORT=8000
GREEN_PORT=8001
UPSTREAM_CONF="${PROJECT_DIR}/nginx/upstream-active.conf"
```

```
# Determine env file (prefer .env.production if it exists)
if [ -f "$PROJECT_DIR/.env.production" ]; then
    ENV_FILE="$PROJECT_DIR/.env.production"
elif [ -f "$PROJECT_DIR/.env" ]; then
    ENV_FILE="$PROJECT_DIR/.env"
else
    echo "ERROR: No .env or .env.production file found"
    exit 1
fi

log() { echo "[$(date -Iseconds)] $*" | tee -a "$LOG_FILE"; }

detect_active_color() {
    if [ ! -f "$UPSTREAM_CONF" ]; then
        log "WARNING: No upstream config found, assuming blue is active"
        echo "blue"
        return
    fi
    if grep -q "127.0.0.1:${GREEN_PORT}.*down" "$UPSTREAM_CONF"; then
        echo "blue"
    elif grep -q "127.0.0.1:${BLUE_PORT}.*down" "$UPSTREAM_CONF"; then
        echo "green"
    else
        log "WARNING: Could not detect active color, assuming blue"
        echo "blue"
    fi
}

port_for_color() {
    if [ "$1" = "blue" ]; then echo "$BLUE_PORT"; else echo "$GREEN_PORT"; fi
}

flip_nginx() {
    local new_active="$1"
    local new_down="$2"
    local new_active_port
    local new_down_port
    new_active_port=$(port_for_color "$new_active")
    new_down_port=$(port_for_color "$new_down")
    log "Flipping nginx: ${new_active} (:${new_active_port}) active, ${new_down} (:${new_down_port}) dou

    cat > "$UPSTREAM_CONF" <<EOF
upstream backend {
    server 127.0.0.1:${new_active_port} max_fails=3 fail_timeout=30s;
    server 127.0.0.1:${new_down_port} down;
    keepalive 32;
}
EOF

    sudo nginx -t && sudo nginx -s reload
    log "Nginx reloaded - traffic now routing to backend-${new_active}"
}
```

```bash
cd "$PROJECT_DIR"

# Acquire deploy lock (prevents concurrent manual + CI deploys)
LOCK_FILE="/tmp/myapp-deploy.lock"
exec 200>"$LOCK_FILE"
if ! flock -n 200; then
    log "ERROR: Another deployment is already running"
    exit 1
fi
trap 'flock -u 200; rm -f "$LOCK_FILE"' EXIT
log "Deploy lock acquired"

DEPLOY_START=$(date +%s)
log "Starting blue-green deployment..."

# 1. Preflight (disk space, docker running, env file valid)
bash "$SCRIPT_DIR/preflight.sh"

# 2. Pull latest code
log "Pulling latest changes..."
git fetch origin main
git checkout main
git pull origin main

# [WARNING] CRITICAL: bash buffers the script in memory. After git pull, this
# shell is still running the OLD script. Re-exec to pick up the new one.
if [ -z "${DEPLOY_REEXEC:-}" ]; then
    export DEPLOY_REEXEC=1
    log "Re-executing with updated deploy script..."
    exec bash "$0"
fi

# 3. Detect current state
ACTIVE_COLOR=$(detect_active_color)
if [ "$ACTIVE_COLOR" = "blue" ]; then NEW_COLOR="green"; else NEW_COLOR="blue"; fi
NEW_PORT=$(port_for_color "$NEW_COLOR")
log "Active: backend-${ACTIVE_COLOR}, deploying to: backend-${NEW_COLOR} (:${NEW_PORT})"

# 4. Build and start the new color
# [WARNING] Pass GIT_REV via --build-arg, NOT via env file. compose YAML substitution
# for build args was unreliable in our testing; --build-arg is guaranteed.
GIT_REV=$(git rev-parse --short HEAD)
export GIT_REV

# Also write to env file for backend runtime env (separate from build args)
if grep -q '^GIT_REV=' "$ENV_FILE"; then
    grep -v '^GIT_REV=' "$ENV_FILE" > "${ENV_FILE}.tmp"
    mv "${ENV_FILE}.tmp" "$ENV_FILE"
fi
echo "GIT_REV=${GIT_REV}" >> "$ENV_FILE"

log "Building backend-${NEW_COLOR} and frontend (git rev: $GIT_REV)..."
docker compose -f "$COMPOSE_FILE" --env-file "$ENV_FILE" build \
    --build-arg "GIT_REV=${GIT_REV}" \
```

```
    --build-arg "VITE_GIT_REV=${GIT_REV}" \
    "backend-${NEW_COLOR}" frontend
docker compose -f "$COMPOSE_FILE" --env-file "$ENV_FILE" up -d "backend-${NEW_COLOR}" frontend

# 5. Wait for health check on new color
log "Waiting for backend-${NEW_COLOR} health check on :${NEW_PORT}..."
for i in $(seq 1 12); do
    if curl -sf "http://127.0.0.1:${NEW_PORT}/health" > /dev/null 2>&1; then
        log "backend-${NEW_COLOR} health check passed!"
        break
    fi
    if [ "$i" -eq 12 ]; then
        log "ERROR: backend-${NEW_COLOR} failed health check after 12 attempts"
        docker compose -f "$COMPOSE_FILE" --env-file "$ENV_FILE" stop "backend-${NEW_COLOR}"
        exit 1
    fi
    sleep 5
done

# 6. Run database migrations
log "Running database migrations..."
docker compose -f "$COMPOSE_FILE" --env-file "$ENV_FILE" exec -T "backend-${NEW_COLOR}" alembic upgrade

# 7. Pre-flip smoke test
# Extract API key from env file for smoke test (your smoke test needs to auth)
SMOKE_TEST_API_KEY=$(grep -E '^WEB_API_KEY=' "$ENV_FILE" 2>/dev/null | head -1 | cut -d= -f2- | tr -d '
export SMOKE_TEST_API_KEY

log "Running pre-flip smoke tests against backend-${NEW_COLOR}..."
if bash "$SCRIPT_DIR/smoke-test.sh" "http://127.0.0.1:${NEW_PORT}"; then
    log "Pre-flip smoke tests passed!"
else
    log "ERROR: Pre-flip smoke tests failed - aborting deploy"
    docker compose -f "$COMPOSE_FILE" --env-file "$ENV_FILE" stop "backend-${NEW_COLOR}"
    exit 1
fi

# 8. Flip traffic
flip_nginx "$NEW_COLOR" "$ACTIVE_COLOR"

# 9. Post-flip smoke test (with auto-rollback on failure)
log "Running post-flip smoke tests..."
if bash "$SCRIPT_DIR/smoke-test.sh" "http://127.0.0.1:${NEW_PORT}"; then
    log "Post-flip smoke tests passed!"
else
    log "ERROR: Post-flip smoke tests FAILED - rolling back"
    echo "[$(date -Iseconds)] ROLLBACK: from=${NEW_COLOR} to=${ACTIVE_COLOR}" >> "${LOG_DIR}/deploy-roll
    flip_nginx "$ACTIVE_COLOR" "$NEW_COLOR"
    docker compose -f "$COMPOSE_FILE" --env-file "$ENV_FILE" stop "backend-${NEW_COLOR}"
    log "Rollback complete. backend-${ACTIVE_COLOR} is serving traffic."
    exit 1
fi

# 10. Verify window (old color stays running in case you need manual rollback)
```

```bash
log "Verification window: ${VERIFY_SECONDS}s"
sleep "$VERIFY_SECONDS"

# 11. Stop old color
log "Stopping backend-${ACTIVE_COLOR}..."
docker compose -f "$COMPOSE_FILE" --env-file "$ENV_FILE" stop "backend-${ACTIVE_COLOR}"

# 12. Verify final state - hit /api/v1/health (NOT /health) to get the git_sha
# [WARNING] /health on the backend returns only {"status":"ok"}. The full health
# JSON with git_sha is at /api/v1/health and requires API key + X-Forwarded-*
# headers to get past the RESTRICT_DIRECT_ACCESS middleware.
HEALTH_API_KEY="$SMOKE_TEST_API_KEY"
HEALTH_RAW=$(curl -sS --max-time 5 \
    -H "X-API-Key: ${HEALTH_API_KEY}" \
    -H "X-Forwarded-For: 127.0.0.1" \
    -H "X-Forwarded-Host: localhost" \
    -H "X-Forwarded-Proto: https" \
    "http://127.0.0.1:${NEW_PORT}/api/v1/health" 2>&1 || echo "__CURL_FAILED__")
log "raw /api/v1/health response: ${HEALTH_RAW}"
DEPLOYED_SHA=$(echo "$HEALTH_RAW" | python3 -c "import sys,json; print(json.load(sys.stdin).get('git_sha
DEPLOY_VERSION=$(cat "$PROJECT_DIR/VERSION" 2>/dev/null || echo "unknown")
DEPLOY_DURATION=$(( $(date +%s) - DEPLOY_START ))
log "Blue-green deployment complete! Active: backend-${NEW_COLOR}, git_sha: $DEPLOYED_SHA"

# 13. Write deploy manifest
MANIFEST_FILE="${PROJECT_DIR}/deploy-manifest.json"
cat > "$MANIFEST_FILE" <<MANIFEST
{
  "git_sha": "${DEPLOYED_SHA}",
  "version": "${DEPLOY_VERSION}",
  "timestamp": "$(date -Iseconds)",
  "deployer": "${DEPLOYER:-manual}",
  "active_color": "${NEW_COLOR}",
  "previous_color": "${ACTIVE_COLOR}",
  "deploy_duration_seconds": ${DEPLOY_DURATION}
}
MANIFEST

# [WARNING] Validate the manifest parses as JSON. If the heredoc produced garbage
# (shell quoting bug, unexpected characters in git metadata), fail the deploy
# so the existing rollback path triggers.
if ! python3 -m json.tool "$MANIFEST_FILE" > /dev/null 2>&1; then
    log "ERROR: deploy-manifest.json is not valid JSON - aborting"
    cat "$MANIFEST_FILE" | tee -a "$LOG_FILE"
    exit 1
fi
log "Deploy manifest written to $MANIFEST_FILE (valid JSON)"

# 14. Cleanup
docker image prune -f
```

4.3 The smoke test script

Create `deploy/smoke-test.sh`:

```bash
#!/usr/bin/env bash
# End-to-end smoke test. Creates and reads a real timer to prove the full
# stack (API + DB + cache) is working.
set -euo pipefail

BASE_URL="${1:-http://localhost:8000}"
API_URL="${BASE_URL}/api/v1/timers"  # CUSTOMIZE: your API path
CLEANUP_PHRASE=""

AUTH_HEADER=()
if [ -n "${SMOKE_TEST_API_KEY:-}" ]; then
    AUTH_HEADER=(-H "X-API-Key: ${SMOKE_TEST_API_KEY}")
fi

# [WARNING] Backend has a RESTRICT_DIRECT_ACCESS middleware that rejects direct-to-
# backend requests unless X-Forwarded-Host matches what nginx would send.
PROXY_HEADERS=(
    -H "X-Forwarded-For: 127.0.0.1"
    -H "X-Forwarded-Host: localhost"
    -H "X-Forwarded-Proto: https"
)

cleanup() {
    if [ -n "$CLEANUP_PHRASE" ]; then
        curl -sf -X DELETE "${API_URL}/${CLEANUP_PHRASE}" "${AUTH_HEADER[@]}" "${PROXY_HEADERS[@]}" > /d
    fi
}
trap cleanup EXIT

echo "=== Smoke Test: ${BASE_URL} ==="

# Health check (nginx-level, no auth)
echo -n "  Health check... "
curl -sf --max-time 5 "${BASE_URL}/health" > /dev/null || { echo "FAIL"; exit 1; }
echo "OK"

# Create a resource
echo -n "  Create resource... "
RESPONSE=$(curl -s -w "\n%{http_code}" --max-time 10 \
    -X POST "${API_URL}" \
    "${AUTH_HEADER[@]}" "${PROXY_HEADERS[@]}" \
    -H "Content-Type: application/json" \
    -d '{"field": "value"}')  # CUSTOMIZE for your API
BODY=$(echo "$RESPONSE" | head -n-1)
CODE=$(echo "$RESPONSE" | tail -n1)
if [ "$CODE" != "201" ] && [ "$CODE" != "200" ]; then
    echo "FAIL (HTTP $CODE: $BODY)"
    exit 1
fi
CLEANUP_PHRASE=$(echo "$BODY" | python3 -c "import sys,json; print(json.load(sys.stdin)['id'])")
echo "OK"

# Read the resource back
echo -n "  Read resource... "
```

```
curl -sf --max-time 10 "${API_URL}/${CLEANUP_PHRASE}" "${AUTH_HEADER[@]}" "${PROXY_HEADERS[@]}" > /dev/n
echo "OK"

echo "=== All smoke tests passed ==="
```

Pattern notes:

- **Smoke tests create AND read** — covers write path, read path, serialization, auth, and DB round-trip.
- **Cleanup on EXIT** — no orphaned test data if the test fails.
- **Forwarded headers** — see the gotcha in Appendix A.
- **API key extracted from env file** at the caller (deploy-bluegreen.sh), passed as env var.

4.4 Preflight check

Create `deploy/preflight.sh`:

```
#!/usr/bin/env bash
# Pre-deploy sanity checks. Abort if anything is obviously broken.
set -euo pipefail

echo "Preflight checks:"

# 1. Docker is running
if ! docker ps > /dev/null 2>&1; then
    echo "  [NO] Docker is not running"
    exit 1
fi
echo "  [YES] Docker is running"

# 2. Disk space (5 GB minimum)
AVAIL=$(df -BG / | awk 'NR==2 {print int($4)}')
if [ "$AVAIL" -lt 5 ]; then
    echo "  [NO] Low disk space: ${AVAIL}GB available, 5GB required"
    exit 1
fi
echo "  [YES] Disk space OK (${AVAIL}GB available)"

# 3. Env file exists and has required vars
if [ ! -f .env.production ] && [ ! -f .env ]; then
    echo "  [NO] No .env or .env.production file"
    exit 1
fi
echo "  [YES] Env file exists"

for var in POSTGRES_PASSWORD REDIS_PASSWORD SECRET_KEY; do
    if ! grep -q "^${var}=" .env.production 2>/dev/null && ! grep -q "^${var}=" .env 2>/dev/null; then
        echo "  [NO] Missing required env var: ${var}"
        exit 1
    fi
    echo "  [YES] ${var} set in env file"
done
```

4.5 Nginx upstream config

Your production nginx config needs to include a file that blue-green writes. Put this in your nginx config:

```
# Inside your server block
include /opt/myapp/nginx/upstream-active.conf;

location /api/ {
    proxy_pass http://backend;  # 'backend' is the upstream defined in upstream-active.conf
    # ... your usual proxy headers
}
```

And make sure `nginx/upstream-active.conf` exists with a sensible default:

```
upstream backend {
    server 127.0.0.1:8000 max_fails=3 fail_timeout=30s;
    server 127.0.0.1:8001 down;
    keepalive 32;
}
```

Blue is the default active color. Deploy-bluegreen.sh will flip this file and reload nginx.

[OK] Phase 4 verification

On your production server:

```
# First deploy (nothing to roll back from)
sudo ./deploy/deploy-bluegreen.sh

# Should see:
# [timestamp] Starting blue-green deployment...
# [timestamp] Building backend-green and frontend (git rev: abc1234)...
# [timestamp] Pre-flip smoke tests passed!
# [timestamp] Flipping nginx: green (:8001) active, blue (:8000) down
# [timestamp] Post-flip smoke tests passed!
# [timestamp] Blue-green deployment complete! Active: backend-green, git_sha: abc1234
# [timestamp] Deploy manifest written to /opt/myapp/deploy-manifest.json (valid JSON)

# Verify manifest
cat deploy-manifest.json

# Verify /api/v1/health reports the real SHA
curl -sS -H "X-API-Key: ..." http://127.0.0.1:8001/api/v1/health | jq .git_sha
# Should match git rev-parse --short HEAD
```

Then test the rollback path: temporarily break your smoke-test.sh (make it exit 1 always), trigger a deploy, and confirm:

- Deploy starts
- Builds new color
- Pre-flip smoke test fails → aborts before flip
- Old color is still serving
- Nothing was broken

Fix your smoke test and deploy again to confirm normal path still works.

Phase 5: Automated Release

Goal: pushing to main automatically opens a Release PR. Merging the Release PR creates a GitHub Release. The GitHub Release automatically triggers a production deploy.

Time: 1–2 hours (plus debugging the gotchas, which is the whole point)

5.1 release-please config

Create `release-please-config.json` at the repo root:

```
{
  "packages": {
    ".": {
      "release-type": "simple",
      "bump-minor-pre-major": true,
      "bump-patch-for-minor-pre-major": true,
      "extra-files": [
        "VERSION",
        "frontend/package.json"
      ],
      "changelog-path": "CHANGELOG.md"
    }
  },
  "$schema": "https://raw.githubusercontent.com/googleapis/release-please/main/schemas/config.json"
}
```

And `.release-please-manifest.json`:

```
{
  ".": "0.1.0"
}
```

This has to match your current VERSION file value.

> [WARNING] **DO NOT put `target-branch: main` in this config file.** It's silently ignored by release-please v16+. `target-branch` is an **action input** in the workflow, not a config file key. This is one of four scoping gaps that the case-study project hit on its first release-please rollout — all four are documented in Appendix A. See entry #3.

5.2 release.yml workflow

`.github/workflows/release.yml`:

```
name: Release

on:
  push:
    branches: [main]

permissions:
  contents: write
  pull-requests: write

jobs:
  release-please:
    name: Release Please
    runs-on: ubuntu-latest
    outputs:
      release_created: ${{ steps.release.outputs.release_created }}
      tag_name: ${{ steps.release.outputs.tag_name }}

    steps:
```

```yaml
    - uses: googleapis/release-please-action@<PIN_TO_LATEST_SHA> # v4+
      id: release
      with:
        config-file: release-please-config.json
        manifest-file: .release-please-manifest.json
        target-branch: main
        token: ${{ secrets.RELEASE_PAT }}  # [WARNING] Critical - see gotcha below
```

[WARNING] **Three gotchas all in this one job, all of which the case-study project hit on the first try:**

Gotcha A: use `googleapis/release-please-action` NOT `google-github-actions/release-please-action`. The latter is deprecated. Use `gh release list --repo googleapis/release-please-action` to get the latest SHA, pin it.

Gotcha B: set `target-branch: main` as an action INPUT (as above). Do NOT put it in `release-please-config.json`. It's silently ignored there.

Gotcha C: Use a Personal Access Token (`secrets.RELEASE_PAT`), NOT the default `GITHUB_TOKEN`. **Releases created via `GITHUB_TOKEN` do not trigger downstream workflows** (intentional GitHub security feature to prevent recursive chains). If you use `GITHUB_TOKEN`, release-please will create the Release successfully, but your `deploy.yml` (triggered by `release: [published]`) will NEVER fire. You'll be dispatching deploys manually forever. Create a PAT with `contents: write` scope and add it as the `RELEASE_PAT` secret.

Also critical: enable the GitHub repo setting "Allow GitHub Actions to create and approve pull requests":

Settings → Actions → General → Workflow permissions → check "Allow GitHub Actions to create and approve pull requests" → Save.

Without this, release-please will fail at its last step with "GitHub Actions is not permitted to create or approve pull requests." You cannot automate this click.

5.3 deploy.yml — the production deploy trigger

`.github/workflows/deploy.yml`:

```yaml
name: Deploy

on:
  release:
    types: [published]

concurrency:
  group: deploy
  cancel-in-progress: false

permissions:
  contents: read

jobs:
  notify-start:
    name: Deploy Started Notification
    runs-on: ubuntu-latest
    steps:
      - name: Send Slack "deploy started" notification
        if: env.DEPLOY_WEBHOOK_URL != ''
        env:
```

```yaml
        DEPLOY_WEBHOOK_URL: ${{ secrets.DEPLOY_WEBHOOK_URL }}
        SLACK_SHA: ${{ github.sha }}
        SLACK_REF: ${{ github.ref_name }}
        SLACK_URL: ${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id }}
      run: |
        PAYLOAD=$(jq -n \
          --arg sha "$SLACK_SHA" \
          --arg ref "$SLACK_REF" \
          --arg url "$SLACK_URL" \
          '{
            blocks: [
              {type: "header", text: {type: "plain_text", text: ":rocket: Deploy Started"}},
              {type: "section", fields: [
                {type: "mrkdwn", text: ("*Commit:*\n`" + $sha + "`")},
                {type: "mrkdwn", text: ("*Ref:*\n`" + $ref + "`")}
              ]},
              {type: "actions", elements: [
                {type: "button", text: {type: "plain_text", text: "View Run"}, url: $url}
              ]}
            ]
          }')
        curl -sf -X POST "$DEPLOY_WEBHOOK_URL" \
          -H "Content-Type: application/json" \
          -d "$PAYLOAD" || echo "::warning::Slack notification failed (non-fatal)"

  test:
    needs: notify-start
    if: ${{ !cancelled() }}   # [WARNING] Don't block deploys on notify-start failures
    uses: ./.github/workflows/ci.yml   # reuse the full CI suite

  deploy:
    name: Deploy to Production
    runs-on: ubuntu-latest
    needs: test
    if: ${{ !cancelled() }}
    environment: production

    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@0ff4204d59e8e51228ff73bce53f80d53301dee2 # v1.2.5
        with:
          host: ${{ secrets.VPS_HOST }}
          username: ${{ secrets.VPS_USER }}
          key: ${{ secrets.SSH_PRIVATE_KEY }}
          port: ${{ secrets.VPS_PORT }}
          script: |
            cd /opt/myapp
            DEPLOYER=ci BLUEGREEN_VERIFY_SECONDS=60 bash deploy/deploy-bluegreen.sh

  notify:
    name: Deploy Notification
    runs-on: ubuntu-latest
    needs: deploy
    if: always()
```

```yaml
    steps:
      - name: Send Slack notification (success or failure)
        if: env.DEPLOY_WEBHOOK_URL != ''
        env:
          DEPLOY_WEBHOOK_URL: ${{ secrets.DEPLOY_WEBHOOK_URL }}
          SLACK_SHA: ${{ github.sha }}
          SLACK_REF: ${{ github.ref_name }}
          SLACK_URL: ${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id }}
          DEPLOY_RESULT: ${{ needs.deploy.result }}
        run: |
          if [ "$DEPLOY_RESULT" = "success" ]; then
            EMOJI=":white_check_mark:"
            TITLE="Deploy Succeeded"
          else
            EMOJI=":x:"
            TITLE="Deploy Failed"
          fi
          PAYLOAD=$(jq -n \
            --arg sha "$SLACK_SHA" \
            --arg ref "$SLACK_REF" \
            --arg url "$SLACK_URL" \
            --arg emoji "$EMOJI" \
            --arg title "$TITLE" \
            '{
              blocks: [
                {type: "header", text: {type: "plain_text", text: ($emoji + " " + $title)}},
                {type: "section", fields: [
                  {type: "mrkdwn", text: ("*Commit:*\n`" + $sha + "`")},
                  {type: "mrkdwn", text: ("*Ref:*\n`" + $ref + "`")}
                ]},
                {type: "actions", elements: [
                  {type: "button", text: {type: "plain_text", text: "View Run"}, url: $url}
                ]}
              ]
            }')
          curl -sf -X POST "$DEPLOY_WEBHOOK_URL" \
            -H "Content-Type: application/json" \
            -d "$PAYLOAD" || echo "::warning::Slack notification failed (non-fatal)"
```

Key patterns:

- **`needs: notify-start` + `if: ${{ !cancelled() }}`** — the deploy depends on the start notification for ordering, but a failed notification doesn't block the deploy. Slack availability is never a hard gate on shipping.
- **`uses: ./.github/workflows/ci.yml`** — reuses the full CI suite via `workflow_call`. No duplicate test logic.
- **`environment: production`** — lets you add protection rules in GitHub (required approvals, deployment windows) without touching the workflow.

5.4 deploy-only.yml — emergency manual dispatch

Always have a way to deploy without going through the release process. Call this the "emergency path."

`.github/workflows/deploy-only.yml`:

```yaml
name: Deploy Only (Skip Tests)

on:
  workflow_dispatch:
    inputs:
      reason:
        description: 'Reason for skipping tests'
        required: true
        type: string

concurrency:
  group: deploy
  cancel-in-progress: false

permissions:
  contents: read

jobs:
  # Same notify-start, deploy, notify structure as deploy.yml above
  # BUT: deploy job does NOT depend on `test` - it goes straight to SSH
  notify-start: # ... same as deploy.yml
  deploy:
    needs: notify-start
    if: ${{ !cancelled() }}
    # ... same SSH step
  notify:  # ... same as deploy.yml
```

When to use:

- Doc-only changes you don't want to run the full test suite for
- Emergency hotfixes where CI is the bottleneck
- The situation described in Appendix A #5 — when `GITHUB_TOKEN` releases don't trigger `deploy.yml` and you need the manual path until your PAT-based release config is in place

[OK] Phase 5 verification

1. Make a feature commit on develop (`git commit -m "feat: test release automation"`)
2. Merge develop → main
3. **Watch**: push to main triggers `release.yml`
4. Wait ~2 minutes — a new PR titled `chore(main): release X.Y.Z` should appear against main
5. Merge that PR
6. **Watch**: merging triggers release-please to create a GitHub Release
7. The GitHub Release publication triggers `deploy.yml`
8. `deploy.yml` runs full CI → deploys → notifies
9. Check Slack — you should see THREE messages: Deploy Started, (eventually) Deploy Succeeded
10. Visit your production site — should be serving the new code

If `deploy.yml` didn't fire: you're hitting Appendix A entry #5. Your release-please is using `GITHUB_TOKEN` instead of a PAT. Fix it before continuing.

Phase 6: Notifications

Goal: Slack (or whatever) gets a real-time trail of your pipeline: CI failures, deploy start, deploy success, deploy failure, weekly health digest.

Most of this is already wired into Phases 3, 4, and 5. This phase is about getting the patterns right.

6.1 Slack mrkdwn is NOT standard Markdown

Slack's message format is called "mrkdwn" (lowercase, one word). It's similar to Markdown but with critical differences:

Does NOT support:

- `#` / `##` / `###` headers (these render as literal text)
- `-` or `*` bullet lists in most contexts
- Blockquotes
- Tables

Supports:

- `*bold*` (single asterisks)
- `_italic_`
- `~strikethrough~`
- `` `code` ``
- Emoji codes like `:rocket:`
- Block Kit JSON (this is how you get real structure)

Lesson: if you need structured messages (headers, fields, multiple sections), use Slack's Block Kit JSON. Don't try to build a pretty markdown string and hope Slack renders it — it won't.

6.2 Use jq -n --arg for JSON, never sed

All of the Slack payloads in this playbook use this pattern:

```
PAYLOAD=$(jq -n \
    --arg sha "$SLACK_SHA" \
    --arg ref "$SLACK_REF" \
    '{
      blocks: [
        {type: "header", text: {type: "plain_text", text: ":rocket: Deploy Started"}},
        {type: "section", text: {type: "mrkdwn", text: ("*Commit:* " + $sha)}}
      ]
    }')
```

Why NOT `sed`:

A more common pattern you'll see in older GitHub Actions examples is:

```
# WRONG - vulnerable to injection
PAYLOAD=$(cat <<'EOF'
{"text": "REPLACE_REASON"}
EOF
)
PAYLOAD=$(echo "$PAYLOAD" | sed "s|REPLACE_REASON|${{ inputs.reason }}|")
```

This breaks if `inputs.reason` contains a pipe character (`|`). An oncall engineer typing `"database issue | hotfix"` in a `workflow_dispatch` reason field will break sed, fail the job, and potentially block the emergency deploy (if `notify-start` is a prerequisite). **This exact bug was caught in code review on the case-study project before it reached production**, but only because a reviewer asked "what does sed do if the reason has a `|` in it?" Use jq.

Also: pass values through `env:` block vars, not direct `${{ }}` interpolation into shell:

```
- name: Notify
  env:
```

```yaml
    SLACK_REASON: ${{ inputs.reason }}  # Safely into an env var
  run: |
    PAYLOAD=$(jq -n --arg reason "$SLACK_REASON" '{text: $reason}')
    # ... jq handles escaping correctly for any value
```

This pattern is injection-safe for arbitrary input, including quotes, newlines, backslashes, and Unicode.

6.3 Weekly pipeline digest (optional but recommended)

Add `.github/workflows/pipeline-digest.yml`:

```yaml
name: Weekly Pipeline Health Digest

on:
  schedule:
    - cron: '0 18 * * 0'  # Sunday 18:00 UTC
  workflow_dispatch:

permissions:
  contents: read
  actions: read

jobs:
  digest:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1

      - name: Gather metrics
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          REPO="${{ github.repository }}"
          SINCE=$(date -u -d "7 days ago" +%Y-%m-%dT%H:%M:%SZ)
          CI_RUNS=$(gh api "repos/${REPO}/actions/workflows/ci.yml/runs?created=>=${SINCE}&per_page=100
          CI_SUCCESS=$(gh api "repos/${REPO}/actions/workflows/ci.yml/runs?created=>=${SINCE}&status=su
          DEPLOY_RUNS=$(gh api "repos/${REPO}/actions/workflows/deploy.yml/runs?created=>=${SINCE}&per_
          {
            echo "CI_RUNS=${CI_RUNS}"
            echo "CI_SUCCESS=${CI_SUCCESS}"
            echo "DEPLOY_RUNS=${DEPLOY_RUNS}"
          } >> "$GITHUB_ENV"

      - name: Send Slack digest
        if: env.DEPLOY_WEBHOOK_URL != ''
        env:
          DEPLOY_WEBHOOK_URL: ${{ secrets.DEPLOY_WEBHOOK_URL }}
        run: |
          PAYLOAD=$(jq -n \
            --arg ci_runs "$CI_RUNS" \
            --arg ci_success "$CI_SUCCESS" \
            --arg deploy_runs "$DEPLOY_RUNS" \
            '{
              blocks: [
                {type: "header", text: {type: "plain_text", text: ":bar_chart: Weekly Pipeline Digest"}
```

```
            {type: "section", fields: [
              {type: "mrkdwn", text: ("*CI runs:*\n" + $ci_runs + " (" + $ci_success + " passed)")}
              {type: "mrkdwn", text: ("*Deploys:*\n" + $deploy_runs)}
            ]}
          ]
        }')
      curl -sf -X POST "$DEPLOY_WEBHOOK_URL" \
        -H "Content-Type: application/json" \
        -d "$PAYLOAD"
```

[OK] Phase 6 verification

- ☐ Trigger a CI failure on main → Slack gets `:rotating_light: CI Failed on main`
- ☐ Trigger a deploy → Slack gets `:rocket: Deploy Started` + `:white_check_mark: Deploy Succeeded`
- ☐ Manually dispatch `pipeline-digest.yml` via GitHub UI → Slack gets a weekly digest
- ☐ Try a workflow_dispatch on deploy-only.yml with `reason="test | with pipe"` — the job should succeed and Slack should render the pipe character correctly

---

Phase 7: Observability

Goal: you can answer "is the app healthy?" in under 30 seconds, and "what changed between this deploy and the last one?" in under 2 minutes.

Time: 1-2 hours

7.1 The two health endpoints

Your backend should expose TWO health endpoints:

`/health` — public, lightweight, no auth. Used by:

- Nginx upstream health checks
- Load balancers
- The post-deploy verification curl in deploy-bluegreen.sh

Returns: `{"status": "ok"}`. That's it. Fast. No database round-trips. Should respond in < 10ms.

`/api/v1/health` — authenticated, full. Used by:

- Monitoring systems
- Dashboards
- Post-deploy "what did I deploy" verification

Returns:

```
{
  "status": "ok",
  "version": "1.4.2",
  "git_sha": "3f2755c",
  "api_version": "v1",
  "database": "connected",
  "cache": "connected"
}
```

> [WARNING] **THIS DISTINCTION MATTERS.** If your deploy verification hits `/health` expecting the full response, you'll get `{"status": "ok"}` back, parse it looking for `git_sha`, fail to find the field, and fall back to `"unknown"`. You'll think the deploy broke the git_sha propagation when actually the deploy is fine and your verification curl is asking the wrong

endpoint. **This is the case-study project's most expensive bug — we nicknamed it *The Health Check That Wasn't.*** Three deploys spent chasing a missing build SHA through every Docker mechanism we had — shell export, compose env file, compose override, `--build-arg`. On the fourth round somebody finally dumped the raw response from the verification curl. It was hitting `/health`, which has never had a `git_sha` field at all. The producer was never broken. Three days of "fixes" on a hole that wasn't there. *(The Engineer + Agent Playbook §7 generalizes the lesson: verify the actual surface before claiming the work is done. The field note "The Health Check That Wasn't" in Part V tells the story from the collaboration angle.)*

7.2 The /health endpoint (FastAPI example)

```python
# backend/app/api/public.py
from fastapi import APIRouter

router = APIRouter()

@router.get("/health")
async def health():
    return {"status": "ok"}
```

7.3 The /api/v1/health endpoint (FastAPI example)

```python
# backend/app/api/v1/health.py
from fastapi import APIRouter, Depends
from sqlalchemy import text
from app.core.config import settings
from app.core.database import get_db

router = APIRouter()

@router.get("/health")
async def health(db = Depends(get_db)):
    # Verify database connectivity
    try:
        db.execute(text("SELECT 1"))
        db_status = "connected"
    except Exception:
        db_status = "error"

    return {
        "status": "ok",
        "version": settings.version,      # read from VERSION file
        "git_sha": settings.git_rev,       # read from GIT_REV env var
        "api_version": "v1",
        "database": db_status,
    }
```

And the config:

```python
# backend/app/core/config.py
import os
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    git_rev: str = "unknown"
    version: str = "unknown"
```

```python
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        # Read VERSION file at startup
        version_file = os.path.join(os.path.dirname(__file__), "..", "..", "VERSION")
        if os.path.exists(version_file):
            with open(version_file) as f:
                self.version = f.read().strip()

settings = Settings()
```

7.4 Frontend git_sha in the footer

In your React app, display the version and SHA in the footer so you can visually verify what's deployed:

```jsx
// frontend/src/components/Footer.tsx
export function Footer() {
  const version = import.meta.env.VITE_VERSION || 'dev';
  const gitRev = import.meta.env.VITE_GIT_REV || 'unknown';
  return (
    <footer>
      v{version} • rev {gitRev}
    </footer>
  );
}
```

Vite bakes the `VITE_*` env vars into the compiled bundle at build time. This is why the `--build-arg VITE_GIT_REV=...` in Phase 4.2 matters — without it, you'll see `rev unknown` in your live site's footer forever.

7.5 Sentry (optional but recommended)

For error tracking:

```bash
# Backend
pip install sentry-sdk
```

```python
# backend/app/main.py
import sentry_sdk
import os

if os.getenv("SENTRY_DSN"):
    sentry_sdk.init(
        dsn=os.getenv("SENTRY_DSN"),
        environment=os.getenv("ENVIRONMENT", "production"),
        release=os.getenv("GIT_REV", "unknown"),  # tag errors with the deploy
        traces_sample_rate=0.1,
    )
```

Critical: pass `release=os.getenv("GIT_REV")` so Sentry groups errors by deploy. This is what lets you say "this error started happening in deploy 3f2755c" instead of "this error has been happening forever."

Client-side Sentry (applies to both tracks) For frontend projects — and especially for Track B projects where there is no backend to capture errors at all — wire Sentry into the React app via an `ErrorBoundary` so render crashes get reported.

```bash
npm install @sentry/react
```

```
// frontend/src/lib/sentry.ts
import * as Sentry from '@sentry/react'

Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,
  environment: import.meta.env.MODE,
  release: import.meta.env.VITE_VERSION + '@' + import.meta.env.VITE_GIT_REV,
  integrations: [
    Sentry.browserTracingIntegration(),
    Sentry.replayIntegration(),
  ],
  tracesSampleRate: 0.1,        // 10% - enough for performance baselines
  replaysOnErrorSampleRate: 1.0, // 100% replays on error - every bug gets a video
  replaysSessionSampleRate: 0,   // 0% proactive replays - privacy-conscious default
})
```

Wire the React `ErrorBoundary` to `Sentry.captureException` so render crashes are reported, not just thrown:

```
// frontend/src/components/ErrorBoundary.tsx
import { Component, ErrorInfo, ReactNode } from 'react'
import * as Sentry from '@sentry/react'

export class ErrorBoundary extends Component<{ children: ReactNode }, { hasError: boolean }> {
  state = { hasError: false }

  static getDerivedStateFromError() { return { hasError: true } }

  componentDidCatch(error: Error, info: ErrorInfo) {
    Sentry.captureException(error, { contexts: { react: { componentStack: info.componentStack } } })
  }

  render() {
    return this.state.hasError ? <FallbackUI /> : this.props.children
  }
}
```

Wrap the root of your component tree in `<ErrorBoundary>`. Now every unhandled render error reaches Sentry instead of vanishing into a blank screen, and the release tag (`version@git_rev`) lets you attribute every error to the specific deploy that introduced it.

> [TIP] **Sample rate guidance for client-side Sentry.** 10% traces is enough to baseline performance without cost explosion. 100% replays on error means every bug report comes with a session video — invaluable for reproducing user-reported bugs. 0% proactive replays is a privacy-conscious default; turn it up only if your users have consented and your DPA covers it. The case-study Track B project ran these exact rates from v5 onward and the replay-on-error setting paid for itself the first time a user reported a paint interaction bug that turned out to be the cache-hit path issue.

7.6 UptimeRobot (free, 5 minutes to set up)

Sign up at https://uptimerobot.com, add a monitor for `https://your-domain.com/health`, set alerts to Slack webhook. Done. You'll get pinged if the site is down for more than 5 minutes.

[OK] **Phase 7 verification**

```
# /health is lightweight and fast
time curl -sS https://your-domain.com/health
# Expected: {"status":"ok"} in < 100ms

# /api/v1/health is authenticated and has the full payload
curl -sS -H "X-API-Key: $API_KEY" https://your-domain.com/api/v1/health | jq
# Expected: git_sha matches current main tip

# Frontend footer shows real version and SHA
curl -sS https://your-domain.com/assets/index-*.js | grep -oE 'rev [a-f0-9]{7}'
```

7.7 Schedule security audits as a recurring discipline *(both tracks)*

Continuous monitoring catches what continuous monitoring is built to catch. It does not catch what continuous monitoring was never told to look for. A periodic security audit — calendar-scheduled, not incident-driven — is the discipline that finds the gaps your normal review cycles have walked past. The Track B case study has two clean examples within a few releases of each other:

- **The v8.1 audit** found CORS configured to `*` in a production fallback, admin checks reading from user-writable `user_metadata` instead of `app_metadata`, missing security headers (CSP, X-Frame-Options, HSTS), and several PostgREST filter injection vectors via `.or()` query builders with user input. None of these were active incidents. All of them had been sitting in the codebase for months.
- **The v9.0 audit** found lost-update race conditions in the voting system (two simultaneous votes producing one result), TOCTOU gaps in entity creation (two simultaneous submissions of "CAT" and "CATS" both passing the existence check), missing UUID validation on admin endpoints (malformed IDs reaching the database), entity-name regex injection vectors used by lookup but not submit, and missing HSTS headers. Again — none were active incidents. The audit was the discovery mechanism.

Both passes generated more fix work than any feature release that quarter, and every fix shipped without an incident driving it. That's the value of the discipline: the audit converts latent risk into closed pull requests on a schedule of your choosing, instead of converting it into incidents on a schedule of your attacker's choosing.

How to run one in practice:

1. **Pick a surface.** Auth + RLS, input validation + injection, dependency vulnerabilities, secrets handling, security headers, rate limiting + DoS, error message information disclosure. One surface per audit; deeper is better than wider.
2. **Schedule it.** Quarterly is a good default for a small team. After every major release if the surface changes a lot. Calendar entry, not "when we get around to it."
3. **Walk every entry point on the chosen surface.** Every endpoint, every middleware, every database policy, every config file. With no goal except *find what's wrong.*
4. **Open issues for every finding, not PRs.** Triage them by severity afterwards and let the next sprint absorb the high-severity ones. The audit itself is the *finding* phase, not the *fixing* phase — separating those phases keeps the audit honest.
5. **Write it up in a retro.** What the surface was, what you found, what you fixed, what you deferred. The retro is the audit trail and the institutional memory.

   [TIP] **Audits don't replace continuous monitoring; they complement it.** Continuous monitoring catches the regressions you've already learned to look for. Audits catch the bug classes you haven't yet learned exist on your project. Most teams have lots of the first kind and none of the second; the second is where the high-value findings live.

   [WARNING] **Don't run an audit without scheduling the follow-up work.** A list of fifteen security findings sitting in a Linear backlog with nobody assigned and no deadline is worse than no audit, because it gives the team a false sense of "we know about it" without any of the

protection that comes from actually fixing it. Schedule the audit and the remediation sprint at the same time. *Pin: an unaddressed audit finding is a vulnerability with documentation.*

Phase 8: Documentation

Goal: a future-you or a new engineer can understand and operate this pipeline without having to reverse-engineer anything.

Time: 2-4 hours, ongoing

8.1 docs/CI-CD.md — the canonical pipeline reference

This playbook you're reading now is the "how to build it" document. `docs/CI-CD.md` should be the "how does it work" document, specific to YOUR repo.

Structure:

```
# CI/CD Pipeline

## Pipeline Overview
[ASCII diagram of the job graph]
[Table of all jobs with what each validates]

## Release Flow
[feature commit → develop → main → release-please → deploy]

## Blue-Green Deployment
[Step-by-step what deploy-bluegreen.sh does]

## Git SHA Propagation
[The backend/frontend split, --build-arg explanation, the /health vs /api/v1/health gotcha]

## Notifications
[Slack Block Kit reference, all notification events, the mrkdwn gotcha]

## Troubleshooting
[All known failure modes with symptoms and fixes]
```

The case-study project's `docs/CI-CD.md` is ~200 lines. It's dry but exhaustive. Do the same.

8.2 docs/OPERATIONS-RUNBOOK.md — for oncall

When production is on fire, nobody has time to read Slack history. The runbook should have:

- **Manual deploy:** exact command to run on the server
- **Rollback:** exact command
- **Deploy lock is stuck:** how to diagnose and unstuck
- **Smoke test fails:** where the logs are, common causes
- **Database is out of connections:** what to do
- **Redis is down:** where graceful degradation kicks in

  [WARNING] **Rule:** any time you delete a script or rename a file, grep the runbook and the docs for references to the old name. If the runbook says "run `deploy/deploy.sh`" and you've deleted that script, you've just bricked the runbook for oncall. The case-study project caught exactly this gotcha during a code review for a deleted legacy script. A reviewer asked "yes, but what else did you break?" and saved a real incident.

8.3 Retrospectives

After each release, write a retro:

- What shipped (bullet list)
- What broke or surprised us
- What we learned
- What's deferred to the next cycle

Retrospectives are MUCH more valuable than changelog entries because they capture the *why* and the *hard-won knowledge*, not just the *what*. Future-you will be grateful. *(The Engineer + Agent Playbook §13 treats the retro as the habit that keeps the human-agent partnership honest — read it if you want the discipline, not just the template.)*

Keep retros in `docs/retrospectives/vX.Y.Z-retro.md`. Write them in your own voice — they're for you, they should be interesting enough to re-read.

8.4 CHANGELOG.md

release-please will auto-generate this as it processes conventional commits. Do NOT edit it manually (except to fix bugs in what release-please generated). If the changelog is wrong, fix your commit messages or release-please config.

8.5 In-repo rules

Create a `CLAUDE.md` (or `.ai-context.md`, or whatever) at the repo root that describes:

- What the project is
- Key architectural decisions
- Common commands
- Task tracking location (Linear / Jira / etc.)

This isn't just for AI agents — it's for you six months from now when you've forgotten why you made a particular decision. The act of writing it is what makes future-work cheaper.

[OK] Phase 8 verification

- ☐ `docs/CI-CD.md` exists and has sections for Overview, Release Flow, Deploy, SHA Propagation, Notifications, Troubleshooting
- ☐ `docs/OPERATIONS-RUNBOOK.md` exists and has Manual Deploy, Rollback, at least 3 troubleshooting entries
- ☐ Any script referenced in the runbook actually exists (`grep -r "deploy.sh" docs/` — any hits should be to real files)
- ☐ `docs/retrospectives/` has an entry for at least your first real release
- ☐ `CHANGELOG.md` exists and was auto-generated by release-please

Track B — PaaS + BaaS

The phases below build the platform-managed track end-to-end. If you're on the self-hosted track instead, see Phase 0 through Phase 8 above. Phase 0 is shared between both tracks. Each Track B phase below explicitly cross-references its Track A counterpart, so you can hop between them when a principle is shared.

Working example: React + Vite frontend on Netlify, with Supabase as the managed PostgreSQL + Auth + RLS backend, GitHub Actions for CI, Playwright (with `@axe-core/playwright`) for E2E, Lighthouse CI for performance and accessibility budgets,

client-side Sentry for error monitoring. The case study is a time-tracking web app shipped through nine major versions in three days. The patterns transfer directly to Vercel + Cloudflare Pages + Render for the host, and to Firebase + PlanetScale + Clerk + Convex + Neon for the BaaS layer. Specific CLI commands change between platforms; the structure of the phases does not.

What you give up vs. Track A: the deploy script as artifact, the ability to SSH into your server, complete control over middleware, the freedom to run arbitrary background processes for free. **What you gain:** the platform handles atomic deploys, instant rollback, blue-green flips, SSL renewal, OS updates, and most of nginx for you — and it does these things well enough that "make the pipeline boring" gets dramatically cheaper. Both trades are real. Pick the one that fits your project.

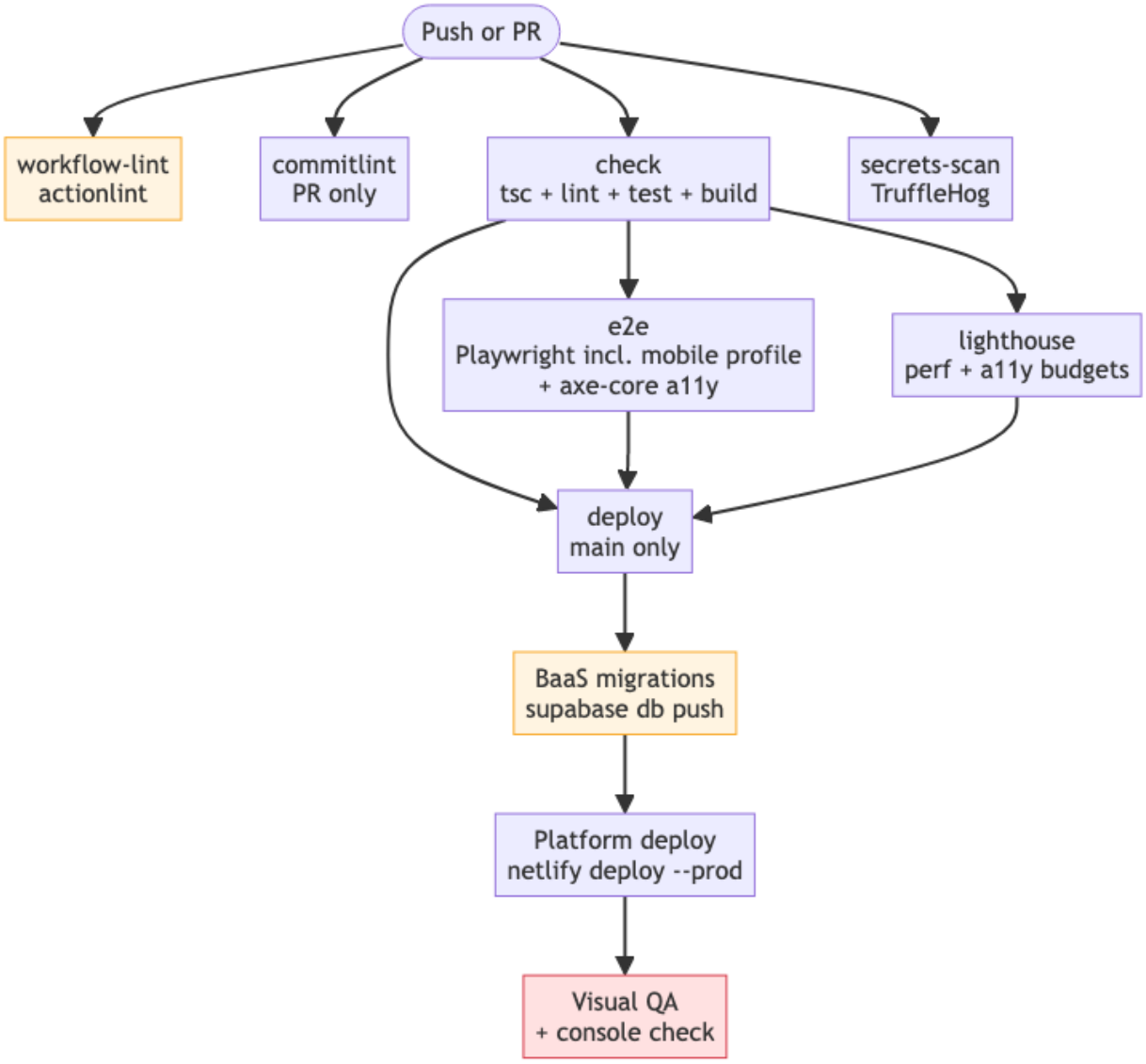
Track B — Pipeline at a Glance

The release chain is shorter on Track B. The platform absorbs most of what Phase 4 does on Track A.

Key transitions:

- [BLUE] `Merge to main` — the trigger. Same as Track A.
- [YELLOW] `BaaS migrations` — **must run before the frontend build deploys.** This is the Track B equivalent of "deploy backend first." If the frontend ships against schema that doesn't exist yet, the new code crashes for every user until the migration lands.
- [GREEN] `Atomic flip` — the platform swaps the live deploy in one operation. Rollback is also one operation, via the platform dashboard. There is no script to write, no nginx config to flip, no health-check loop to manage. The platform owns this.

The CI job graph is similar to Track A's, with a few differences:



Architecture notes:

- [YELLOW] `workflow-lint` and `BaaS migrations` are both gates that catch a class of bug at the right wall. actionlint catches workflow YAML bugs at CI time; the migration step catches schema-vs-code drift at deploy time.
- The `check` job replaces Track A's split between `backend-test` and `frontend-test` — there's no backend code to test in this track, so the single check job runs TypeScript + lint + Vitest + production build.
- [RED] `Visual QA` is the Track B equivalent of Track A's "curl /api/v1/health and read the JSON." There's no backend endpoint to curl. Verification is the platform-managed equivalent of a smoke test: open the live URL, walk every page, check the console.

Track B Phase 0: Foundation

Identical to Track A Phase 0 with one delta. Work the Phase 0 above end-to-end: git-flow with `develop` + `main`, conventional commits via commitlint, branch protection on `main`, `.gitignore` and `.dockerignore` audited (yes, even though Track B doesn't ship a Dockerfile — the BaaS local dev environment is Docker-based, and `.gitignore` matters regardless).

The one delta: Track B doesn't need a separate `VERSION` file at the repo root. The `version` field in `package.json` is the source of truth, and Vite reads it directly at build time (see Track B Phase 1.2 below for the version-stamping pattern). When release-please bumps the version, it bumps `package.json` instead of `VERSION`. Configure release-please's `extra-files` to include `package.json` and you're done.

Track B Phase 1: Local Development

> *This is where Track B diverges sharply from Track A.* Track A Phase 1 builds a multi-stage Dockerfile, a docker-compose.yml for local services, and a docker-compose.prod.yml for the deploy target. Track B has none of those. Instead: Vite handles the build, the BaaS CLI handles local service emulation, and `.env.local` holds the local config.

B1.1 The build is `vite build`. There is no Dockerfile.

Your local dev loop is:

```
npm install
npm run dev          # Vite dev server with HMR on http://localhost:5173
```

Production build:

```
npm run build        # tsc -b && vite build → ./dist
npm run preview      # Serve ./dist locally to verify the production build
```

The production target is the platform's hosting environment, which serves the static `./dist` directory through its CDN. There is no container to build, no nginx to configure, no PM2 to manage.

B1.2 Version stamping with Vite — read `package.json` directly

Track A stamps the git SHA into the Docker image with `--build-arg`. Track B stamps the version and git SHA into the Vite bundle via `define:` in `vite.config.ts` — and the version is read from `package.json` at build time, not from `process.env.npm_package_version` (which is unreliable across npm versions and Vite configurations).

```
// vite.config.ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import { readFileSync } from 'fs'
import { execSync } from 'child_process'
import path from 'path'
import { fileURLToPath } from 'url'

const __dirname = path.dirname(fileURLToPath(import.meta.url))

// Read version directly from package.json - process.env.npm_package_version is unreliable
const pkg = JSON.parse(readFileSync(path.resolve(__dirname, 'package.json'), 'utf-8'))

// Read git rev at build time
let gitRev = 'unknown'
try {
  gitRev = execSync('git rev-parse --short HEAD').toString().trim()
} catch { /* not in a git repo, leave as unknown */ }

export default defineConfig({
  plugins: [react()],
  define: {
    __APP_VERSION__: JSON.stringify(pkg.version),
```

```
    __APP_GIT_REV__: JSON.stringify(gitRev),
  },
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') },
  },
})
```

In your TypeScript code, declare the globals (in `src/vite-env.d.ts`):

```
declare const __APP_VERSION__: string
declare const __APP_GIT_REV__: string
```

And use them directly:

```
// src/components/Footer.tsx
export function Footer() {
  return <footer>v{__APP_VERSION__} • rev {__APP_GIT_REV__}</footer>
}
```

> [WARNING] **Why not `import.meta.env.npm_package_version`?** It's unreliable across npm versions and Vite configurations. The case-study Track B project (v5.0) shipped a release where the footer displayed `v1.0.0` long after they were on v5 — root cause was that `npm_package_version` returned the wrong value depending on which npm command invoked `vite build`. The fix was to read `package.json` directly with `readFileSync` at config-evaluation time. It's deterministic. It always works. See Gotcha Hall of Fame.

B1.3 BaaS local dev — Supabase as the example

Supabase ships a CLI that runs the entire stack (PostgreSQL + Auth + Realtime + Storage) locally in Docker. This is your "local services" layer — the equivalent of Track A's `docker-compose up`.

```
supabase init                  # One-time: creates supabase/ directory
supabase start                 # Start local Supabase (Docker required)
supabase status                # Show URLs and keys for the local instance
supabase stop                  # Shut down

supabase migration new add_categories  # Create a new migration
supabase db reset              # Apply all migrations from scratch (destroys local data)
supabase db push               # Push local migrations to the linked cloud project
```

The local Supabase instance gives you a real PostgreSQL with the same schema, RLS policies, and Auth flows as production. **Dev/prod parity is structural, not aspirational.** A test that passes against `supabase start` will pass against the cloud project, because they're the same Postgres version, the same migration set, the same RLS policies.

For Firebase: `firebase emulators:start` does the equivalent. For PlanetScale: branch databases and `pscale connect`. For Convex: `npx convex dev`. The pattern is the same — the BaaS CLI brings up a local instance that matches production.

B1.4 Environment variables — `.env.local` and `.env.example`

Same pattern as Track A: commit `.env.example`, gitignore `.env.local`. The variable naming convention for Vite is `VITE_*` — anything prefixed with `VITE_` is exposed to client code via `import.meta.env.VITE_*`. **Anything not prefixed with `VITE_` is server-only and will not be bundled.** Use this distinction deliberately.

```
# .env.example - commit this
VITE_SUPABASE_URL=https://your-project.supabase.co
```

```
VITE_SUPABASE_ANON_KEY=your-anon-key-here
VITE_SENTRY_DSN=https://your-dsn@sentry.io/12345
```

> [WARNING] **The Supabase anon key goes in the client bundle. This is intentional.** It's a *publishable key*, protected by Row-Level Security on every table. The thing you must NEVER commit or expose is the **service role key**, which bypasses RLS. See Gotcha Hall of Fame: *Publishable vs. service-role keys on BaaS*.

B1.5 Security headers via platform config

Track A configures security headers in nginx. Track B configures them in the platform's config file. For Netlify:

```
# netlify.toml
[build]
  command = "npm run build"
  publish = "dist"

[[headers]]
  for = "/*"
  [headers.values]
    X-Frame-Options = "DENY"
    X-Content-Type-Options = "nosniff"
    Referrer-Policy = "strict-origin-when-cross-origin"
    Permissions-Policy = "camera=(), microphone=(), geolocation=()"
    Content-Security-Policy = "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline' 

# SPA fallback - serve index.html for all routes
[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

Vercel's equivalent lives in `vercel.json`. Cloudflare Pages uses `_headers` and `_redirects` files. The header values are the same; only the file format differs.

[OK] Track B Phase 1 verification

- ☐ `npm run dev` starts the dev server and you can load the app at the local URL
- ☐ `npm run build` produces a `./dist` directory
- ☐ `npm run preview` serves `./dist` and the production build loads correctly
- ☐ The footer displays the real version from `package.json` and a real git SHA, not `unknown`
- ☐ `supabase start` (or your BaaS equivalent) brings up a local instance and `supabase status` reports it as healthy
- ☐ `.env.example` is committed; `.env.local` is gitignored
- ☐ Security headers configured in `netlify.toml` (or platform equivalent)
- ☐ Curl your local production preview and verify response headers include the security headers you set

Track B Phase 2: Basic CI

> *Mostly shared with Track A Phase 2.* Same `workflow_call`, same SHA-pinning, same commitlint, same secrets-scan, same Lighthouse CI / axe-core / touch-device E2E gates from sections 2.4a / 2.4b / 2.4c above. The differences are: no separate backend-test job, no docker-build job, the check job runs the full frontend build with placeholder env vars.

```
# .github/workflows/ci.yml
name: CI
```

```yaml
on:
  pull_request:
    branches: [main, develop]
  push:
    branches: [main, develop]

permissions:
  contents: read

jobs:
  workflow-lint:
    # Same as Track A Phase 3.1 - actionlint as the FIRST job
    # See Phase 3 above

  commitlint:
    # Same as Track A Phase 2.2

  check:
    name: Lint, Test & Build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1
      - uses: actions/setup-node@49933ea5288caeca8642d1e84afbd3f7d6820020 # v4.4.0
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - name: TypeScript check
        run: npx tsc --noEmit
      - name: Lint
        run: npm run lint
      - name: Unit tests
        run: npm run test
      - name: Production build
        run: npm run build
        env:
          # Placeholders are fine for the build check - the real values come
          # from secrets at deploy time. The build only needs them to *exist*.
          VITE_SUPABASE_URL: ${{ secrets.VITE_SUPABASE_URL || 'https://placeholder.supabase.co' }}
          VITE_SUPABASE_ANON_KEY: ${{ secrets.VITE_SUPABASE_ANON_KEY || 'placeholder-key' }}
      - name: Upload build artifact
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/
          retention-days: 1

  e2e:
    name: E2E Tests
    runs-on: ubuntu-latest
    needs: check
    steps:
```

```yaml
      - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1
      - uses: actions/setup-node@49933ea5288caeca8642d1e84afbd3f7d6820020 # v4.4.0
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run build
        env:
          VITE_SUPABASE_URL: ${{ secrets.VITE_SUPABASE_URL || 'https://placeholder.supabase.co' }}
          VITE_SUPABASE_ANON_KEY: ${{ secrets.VITE_SUPABASE_ANON_KEY || 'placeholder-key' }}
      - run: npx playwright install --with-deps chromium
      - name: Run E2E tests (chromium + mobile-chrome, includes axe-core)
        run: npm run test:e2e
        env:
          VITE_SUPABASE_URL: ${{ secrets.VITE_SUPABASE_URL || 'https://placeholder.supabase.co' }}
          VITE_SUPABASE_ANON_KEY: ${{ secrets.VITE_SUPABASE_ANON_KEY || 'placeholder-key' }}
      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 14

  lighthouse:
    # See section 2.4a above

  secrets-scan:
    # Same as Track A Phase 2.5
```

Notes:

- **Placeholder env vars in the build step.** The build only needs the env vars to *exist*; it doesn't validate them at build time (Supabase reads them at runtime). Use placeholder values when the secrets aren't available, so PRs from forks can still run CI without leaking real keys.
- **Build artifact upload from `main` only.** The `deploy` job (next phase) downloads this artifact instead of rebuilding. Saves CI time and guarantees the same bytes shipped.
- **Lighthouse CI and axe-core E2E are non-negotiable** for Track B — they're how you replace the backend health endpoint as the "did this break anything?" signal. See sections 2.4a–2.4c.

[OK] Track B Phase 2 verification

- ☐ CI runs on every push and PR to `develop` and `main`
- ☐ `workflow-lint` (actionlint) is the first job and passes
- ☐ `check` runs tsc + lint + test + build with placeholder env vars
- ☐ `e2e` runs Playwright on both desktop and mobile profiles, including axe-core a11y assertions
- ☐ `lighthouse` runs `@lhci/cli autorun` and asserts performance + accessibility budgets
- ☐ Build artifact is uploaded from `main` pushes for the deploy job to consume
- ☐ All jobs are SHA-pinned (no `@v4` or `@main`)

Track B Phase 3: CI Self-Protection

> *Mostly shared with Track A Phase 3, with two phases removed.* `actionlint` (Phase 3.1) applies to both tracks unchanged. The Docker context exclusion check and Trivy container scan don't apply — there's no Docker image. The CI failure notification (3.4) applies the same way; use Block Kit and `jq -n --arg` regardless of track.

What replaces the Docker scan: **dependency vulnerability scanning** via `npm audit` (already in Phase 2) plus optional Snyk or Dependabot. For BaaS-side security, the platform handles its own image scanning

— that's part of what you're paying them for.

Track B Phase 4: Atomic Platform Deploy

> *This is where Track B's largest divergence from Track A lives.* Track A Phase 4 builds a 14-step blue-green deploy script with health checks, smoke tests, manifest validation, and auto-rollback. Track B has none of those. The platform owns blue-green, atomicity, and rollback. The deploy job is a CLI call.

B4.1 The deploy job

```yaml
deploy:
  name: Deploy to Netlify
  runs-on: ubuntu-latest
  needs: [check, e2e, lighthouse]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'

  steps:
    - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1

    # STEP 1 - BaaS migrations FIRST. This is the Track B equivalent of
    # Track A's "deploy backend first." The frontend will reference new
    # schema; the schema must exist before the frontend lands.
    - name: Apply Supabase migrations
      run: |
        npx supabase db push --db-url "${{ secrets.SUPABASE_DB_URL }}"

    # STEP 2 - Download the build artifact from the check job
    - name: Download build artifact
      uses: actions/download-artifact@v4
      with:
        name: dist
        path: dist/

    # STEP 3 - Platform deploy. The platform handles blue-green atomically.
    - name: Deploy to Netlify
      run: |
        npx netlify-cli deploy --prod \
          --dir=dist \
          --site=${{ secrets.NETLIFY_SITE_ID }} \
          --auth=${{ secrets.NETLIFY_AUTH_TOKEN }}

    # STEP 4 - Post-deploy verification (see Phase 7)
    - name: Wait for deploy and check live URL
      run: |
        sleep 10  # Give the CDN a moment to propagate
        curl -fsSL https://your-domain.com/ | grep -q "rev ${GITHUB_SHA::7}" \
          || (echo "Footer git SHA mismatch - deploy did not propagate" && exit 1)
```

That's the entire deploy. Three substantive steps and a verification curl.

B4.2 BaaS migrations must run before the frontend deploys

This is the Track B version of Track A's ”deploy backend first, then frontend” rule, and it has the same root cause: schema drift. If the new frontend code references a column that doesn't exist yet, every user gets a

runtime error from the moment the platform flips traffic until the migration lands.

The mitigation: run the migration step *before* the platform deploy step, in the same CI job, with no parallelism between them. If the migration fails, the deploy job aborts and the platform never flips. Your users see the previous version, exactly as if the deploy had never started. This is the Track B equivalent of Track A's pre-flip safe abort — same property, different mechanism.

> [WARNING] **Migration ordering matters even more on Track B than on Track A**, because Track A has nginx as a buffer (you can flip traffic back to the old color if the new color is broken), while on Track B the platform's atomic flip happens in one operation and then you're committed. The escape hatch is the platform's rollback button, which is fast but not instant. The pre-deploy migration step is your Phase 4 "pre-flip smoke test" — if migrations fail, you abort before the flip, and nothing changes for users.

B4.3 Rollback

There is no rollback script to write. The platform provides one-click rollback via its dashboard, and most platforms also expose it via CLI:

```
netlify deploys:list                # See deploy history
netlify rollback <deploy-id>        # Or click "Publish" on a previous deploy in the dashboard
```

For BaaS-side rollback, your migrations should be reversible (write the `down` migration when you write the `up`). Supabase migrations are append-only by default; if you need to undo a schema change, write a new migration that reverses it rather than trying to delete the old one. **Migration history is part of the audit trail, not a thing to edit.**

B4.4 What does NOT apply to Track B

The following Track A Phase 4 gotchas are not relevant to Track B because there is no deploy script and no nginx:

- ☒ Bash self-reexec after `git pull` (no script)
- ☒ Compose env-file vs. `--build-arg` for `GIT_REV` (no docker-compose)
- ☒ Pre-flip and post-flip smoke tests via `curl` (no nginx flip)
- ☒ Deploy lock via `flock` (the platform serializes deploys for you)
- ☒ Manifest validation via `python3 -m json.tool` (the platform's deploy history is the manifest)
- ☒ Verify window with old color still running (rollback is via dashboard, not by keeping containers warm)

These are not "you don't need them" — they are "the platform handles the underlying problem they were solving." The principles (atomicity, rollback, version visibility) all still apply. The mechanics are the platform's, not yours.

B4.5 What DOES apply to Track B

- [OK] **The release event triggers the deploy job** (Phase 5: release-please).
- [OK] **`GITHUB_TOKEN`-created releases don't trigger downstream workflows** — use a PAT for release-please, same as Track A.
- [OK] **Backend-first ordering** — BaaS migrations before frontend deploy, same logical rule as Track A's "migrate before flip."
- [OK] **Post-deploy verification** — hit the live URL and confirm the version footer matches what you just shipped (Phase 7).
- [OK] **Slack notifications** on deploy success/failure (Phase 6).
- [OK] **Every deploy must be answerable** — the platform's deploy history + the version footer + Sentry release tagging is the manifest equivalent.

B4.6 The handler pattern — extract edge function boilerplate before it diverges

Track A's deploy script is the artifact you can audit. Track B's equivalent — the thing that *actually owns* most of your security-relevant code — is your edge function handler layer. CORS handling, auth verification, rate limiting, error shaping, response formatting, request body parsing: every one of these is something every endpoint has to do, and every one of these is something every endpoint will do *slightly differently* if you let each function author wire it up by hand. Before long you have fifteen different CORS headers, twelve different error response shapes, and three endpoints that forgot rate limiting entirely.

The fix is the **handler pattern**: a single shared handler function that accepts a config (declaring the security posture: which auth, which rate limit, which CORS allowlist, which client privilege level) and a business-logic function, and runs the whole pipeline in a known order:

```
// supabase/functions/_shared/handler.ts
export type HandlerConfig = {
  auth: 'required' | 'optional' | 'none'
  client: 'anon' | 'admin'                  // see B4.7 - defaults must be safe
  rateLimit: { window: number; max: number } | 'none'
  cors: { allowOrigins: string[] }
  maxBodyBytes?: number                     // see Phase 2.4e
}

export async function handle(
  req: Request,
  config: HandlerConfig,
  business: (ctx: Context) => Promise<Response>,
): Promise<Response> {
  // 1. CORS preflight
  if (req.method === 'OPTIONS') return corsPreflightResponse(config.cors)

  // 2. Body size limit (early 413)
  if (config.maxBodyBytes && contentLengthExceeds(req, config.maxBodyBytes)) {
    return jsonError(413, 'Payload Too Large')
  }

  // 3. Rate limit (IP-hash for unauth, user-id for auth)
  const limited = await checkRateLimit(req, config.rateLimit)
  if (limited) return jsonError(429, 'Too Many Requests')

  // 4. Auth (verify JWT manually; verify_jwt at gateway is deprecated)
  const ctx = await authenticate(req, config.auth, config.client)
  if (config.auth === 'required' && !ctx.user) return jsonError(401, 'Unauthorized')

  // 5. Business logic
  try {
    return await business(ctx)
  } catch (e) {
    return errorResponseFor(e)              // shared error shape
  }
}
```

Each endpoint then becomes a six-line declaration of *what it is* plus a function for what it does:

```
// supabase/functions/get-leaderboard/index.ts
import { handle } from '../_shared/handler.ts'
```

```
Deno.serve((req) => handle(req, {
  auth: 'none',
  client: 'anon',                          // public read; no admin needed
  rateLimit: { window: 60, max: 30 },
  cors: { allowOrigins: ['https://your-domain.com'] },
}, async (ctx) => {
  const data = await ctx.db.from('leaderboard_view').select('*').limit(100)
  return jsonResponse(200, data)
}))
```

> **Field note — from a Track B case study:** *825 Lines That Were Lying to Each Other.* A web app reached fifteen edge functions, each with its own copy of CORS, auth, rate-limit, and error-shape boilerplate. The total was ~825 lines of duplicated infrastructure code, with subtle differences across copies — two functions had different CORS headers, three had different error response shapes, two had silently *omitted* the rate-limit step entirely. A v8.2 audit found the divergence and extracted everything into a 139-line shared handler pipeline. **The extraction immediately caught the two endpoints that had been missing rate limiting** — not because the audit looked for them, but because the new pattern wouldn't compile without them. The lesson: a duplicated pattern across more than three or four callers will silently drift, and the drift will be in the security-relevant fields you'd most like it not to drift in. Extract the handler before you have fifteen copies that have stopped agreeing with each other. *Pin: shared handlers don't just save lines; they make the security posture of every endpoint legible at a glance.*

B4.7 Default to anon, opt in to admin

The corollary to the handler pattern. When your handler config includes a `client:` field, **the default value must be `'anon'`, and `'admin'` must be an explicit opt-in.** Not the other way around.

Why this matters: defaults are the part of the design that catches every developer who doesn't think about the field — including AI agents working from incomplete context, including reviewers who skim the config block, including future you on a tired afternoon. If the default is `admin`, *not thinking about which client to use* gives you full database privileges. If the default is `anon`, not thinking about it gives you the safe choice and forces you to write the word `admin` explicitly when you actually need elevation.

This is the Track B equivalent of "least privilege" applied to the handler layer. The same principle applies to every other config field: rate limit defaults to *limited* (require explicit `'none'` for the rare exception), CORS defaults to a strict allowlist, auth defaults to `'required'`. Build the safe choice into the type signature so you have to actively choose to weaken it.

> **Field note — from a Track B case study:** *The Default That Was Admin.* A handler refactor introduced a `client:` field defaulting to `'admin'` if the caller didn't specify. Existing endpoints kept working because they had been written before the refactor and explicitly passed the right client. Two new endpoints written *after* the refactor — a public stats endpoint and a public leaderboard — silently shipped with admin privileges because the developers didn't think about the `client:` field at all. RLS happened to cover the data they touched, so no leak occurred. The next security audit caught the inversion and changed the default to `'anon'`. The fix was one line. The lesson is the corollary to "authorization is scoped, not blanket": **the unsafe default is the one that gets used by everyone who didn't think about it. Make the safe one the default; require explicit opt-in for elevation.** Defaults catch the developers (and agents) who didn't read the docs. Defaults must be the safe choice.

B4.8 Pre-deployment configuration ordering — set the prerequisite before deploying the dependency

The Track B equivalent of Track A Phase 4's "preflight check" is *cross-system dependency ordering.* When a deploy adds a startup guard, a runtime check, or a behaviour that requires some external state to already exist (a secret in the vault, a config in the dashboard, a row in a config table, a feature flag set, a third-party

webhook configured), **that prerequisite must be in place before the deploy that depends on it lands.**

The failure mode is mechanical and brutal: the deploy lands, the new code expects the prerequisite, the prerequisite isn't there, the new code refuses to function (or worse, falls back to an unsafe default), and you're either rolling back or scrambling to set the missing prerequisite while the site is down. The fix is always thirty seconds; the outage is always thirty minutes plus however long it takes to figure out what was missing.

The discipline is to **walk the deploy plan and ask, for every change, "what must already be true in some other system before this change can run?"** — then add the prerequisite-creating step as its own task earlier in the plan. This is part of the spec coverage rule from the engineer-agent playbook (§12), but it lands in Phase 4 of the DevOps playbook because the failure mode is *deploys that brick themselves on the prerequisite*, not just *plans that miss requirements*.

A short checklist, runnable before any deploy that touches more than the application code:

1. Does this deploy add a *startup-time check* (env var must exist, file must exist, table must exist)? If yes — is the thing the check requires already in place in the target environment?
2. Does this deploy reference a *new secret*, *new env var*, or *new config*? If yes — has the secret been added to the secrets vault / dashboard?
3. Does this deploy reference a *new database column, table, or function*? If yes — has the migration run, *and* does the deploy job run the migration *before* the application code lands?
4. Does this deploy reference a *new third-party resource* (webhook URL, OAuth callback, API key, DNS record)? If yes — has that resource been registered with the third party?
5. Does this deploy *change the contract* with another system (a downstream service, a frontend bundle, a scheduled job)? If yes — is the other system either backwards-compatible with both versions, or scheduled to update in lockstep?

Walk the list before the deploy, not after.

> **Field note — from a Track B case study:** *The Guard That Came Before the Secret.* A v8.3 release added a startup-time check: if the `IP_SALT` environment variable wasn't set, the API would refuse to boot rather than fall back to a hardcoded default that would silently weaken the IP-hashing scheme. Good change. The PR was clean. Tests passed. Code review approved it. The deploy ran. The new container started. The startup check fired. The container exited. The deploy retried twice and rolled back. Root cause: the *guard* shipped, but the step "set `IP_SALT` in the production secrets vault" was never added to the plan — a teammate had assumed the variable was already there because the name had appeared in earlier code. It hadn't been. Fix was thirty seconds (set the secret, redeploy). Lesson: **a deploy that introduces a precondition must include the step that establishes the precondition's prerequisite, in the right order, in the right system.** Walking the plan against the spec is not enough on its own; you also have to walk the *dependencies between systems* and verify the order of operations across them. *Pin: when a deploy assumes another system is in a particular state, the assumption is part of the deploy plan — and the step that gets the system into that state is part of the deploy plan.*

[OK] Track B Phase 4 verification

- ☐ Merging to `main` triggers the deploy job
- ☐ BaaS migrations run before the platform deploy step
- ☐ If migrations fail, the deploy aborts and the platform never receives the new build
- ☐ The deploy CLI call succeeds and the platform reports the new deploy as live
- ☐ The post-deploy curl confirms the live site's footer git SHA matches the commit SHA
- ☐ You can roll back via the platform dashboard or CLI in under one minute
- ☐ You have written down the rollback procedure in the operations runbook (Phase 8)

Track B Phase 5: Automated Release

> *Largely shared with Track A Phase 5.* release-please works the same way, conventional commits work the same way, all the gotchas from Phase 5 apply (target-branch as action input, PAT not GITHUB_TOKEN, anchor release, etc.).

The deltas:

- **`extra-files` includes `package.json` instead of `VERSION`.** release-please bumps the version field in `package.json` directly. Because Vite reads `package.json` at build time (Track B Phase 1.2), the new version appears in the footer immediately on the next deploy.
- **The deploy is triggered by the same `release: published` event**, but it fires the Track B deploy job from Phase B4 instead of a blue-green script.

```yaml
# .github/workflows/release-please.yml
name: Release Please
on:
  push:
    branches: [main]

permissions:
  contents: write
  pull-requests: write

jobs:
  release-please:
    runs-on: ubuntu-latest
    steps:
      - uses: googleapis/release-please-action@<sha>
        with:
          token: ${{ secrets.RELEASE_PLEASE_PAT }}  # NOT GITHUB_TOKEN
          target-branch: main                       # action input, NOT config key
          release-type: simple
          bump-minor-pre-major: true
          extra-files: |
            package.json
```

Everything else from Phase 5 — the gotcha hall of fame entries 3, 4, 5 in Appendix A — applies to Track B unchanged.

Track B Phase 6: Notifications

> *Identical to Track A Phase 6.* Slack Block Kit (not mrkdwn), `jq -n --arg` for JSON construction (not sed), CI failure notifications on `main` only. Use the same patterns. The events being notified are slightly different (no blue-green flip events), but the templates and the gotchas are the same.

Track B Phase 7: Observability

> *This is the second-largest divergence from Track A.* Track A Phase 7 builds two health endpoints (`/health` and `/api/v1/health`) and uses `curl` against them as the verification primitive. Track B has no backend code to host a health endpoint — the "health surface" is the rendered application plus the BaaS provider's status.

B7.1 What replaces the backend health endpoint

There is no `/api/v1/health` to curl on Track B. The "is this thing alive and serving the right version?" question is answered by:

1. **The footer on every page** displays the real version + git SHA via the Vite `define:` mechanism (Phase 1.2). This is your "live SHA" check — you can `curl https://your-domain.com/ | grep "rev abc1234"` and confirm the deployed bundle matches the commit you just merged.
2. **The platform's deploy history** is your manifest. Netlify, Vercel, Cloudflare Pages all keep an audit log of every deploy with timestamp, git SHA, deployer, and a link to the live deploy. This replaces Track A's `deploy-manifest.json` file on the server.
3. **The BaaS provider's status page** (status.supabase.com, status.firebase.google.com, etc.) is your dependency health signal. UptimeRobot can monitor the BaaS status page just like it monitors the public URL.
4. **Sentry releases tagged with version + git rev** (Phase 7.5) attribute every error to the deploy that introduced it. *"Errors started after release 9.0.0+abc1234"* is a one-glance answer to "did this deploy break something?"

B7.2 Post-deploy verification — the visual QA matrix

Track A's post-deploy verification is `curl /api/v1/health` and read the JSON. Track B's is **a structured visual QA matrix** — every page × every theme × every viewport × zero console errors as a hard gate.

The case-study Track B project shipped this table in their v4.0 retro:

Page	Dark	Light	Mobile (375px)
Login	Pass	Pass	Pass
Signup	Pass	Pass	Pass
Reset Password	Pass	Pass	—
Day View	Pass	Pass	Pass
Week View	Pass	Pass	Pass
Analytics	Pass	Pass	Pass
Category Manager	Pass	—	—

Plus three hard gates:

- Zero console errors across all pages
- All security headers verified via `curl -sI https://your-domain.com/`
- One full real interaction flow tested end-to-end (sign up, perform the core action, sign out)

This is your smoke test. It looks softer than `curl` because it involves a human walking through pages, but it catches a class of bug `curl` cannot — render regressions, theme bugs, mobile layout breaks, hover-only affordances disappearing on touch, console errors that don't fail the build but do break things downstream. The visual QA matrix template is in Appendix B.

You can partially automate this with Playwright + axe-core (Phase 2.4b–2.4c), and you should. But the final visual pass on a real browser before declaring a release done is non-negotiable for Track B, because the alternative is a green CI run that nobody actually looked at.

B7.3 Performance and accessibility as ongoing observability

On Track B, Lighthouse CI (Phase 2.4a) is not just a CI gate — it's also your ongoing performance and accessibility observability. Run Lighthouse on every push, store the reports as CI artifacts with 14-day retention, and you have a longitudinal record of how your scores are drifting. Performance regressions become visible as soon as they enter `develop`, not after they've reached production.

This is the Track B equivalent of Track A's per-deploy metrics. Same goal — *"how is the system actually behaving over time?"* — different signal source.

B7.4 Client-side Sentry with ErrorBoundary

See Phase 7.5 above — the "Client-side Sentry" subsection covers the React + ErrorBoundary integration. Track B reads the same subsection. The release tag should be `${__APP_VERSION__}@${__APP_GIT_REV__}` so every error is attributed to the deploy that introduced it.

For Track B specifically, client-side Sentry is the *only* place runtime errors get captured — there is no backend Sentry. Wire it on day one.

B7.5 UptimeRobot for the public URL and the BaaS status page

Same setup as Track A Phase 7.6. Add two monitors instead of one:

1. `https://your-domain.com/` — the live frontend
2. `https://status.your-baas.com/` (or the BaaS status page URL) — the dependency status

If either goes down, you get paged. The frontend going down means your platform host is having a problem; the BaaS going down means your data layer is. Both are good to know.

B7.6 SPA SEO via meta-tag injection at the edge

A pure SPA is invisible to crawlers. Every route returns the same `index.html` with the same generic meta tags, because the routes only become real after the JavaScript runs — and most crawlers don't run JavaScript well, or at all. If your project has any need for organic search traffic, social link previews, or per-route Open Graph cards, the static `index.html` approach silently destroys all of it.

The Track B answer is **edge-side meta tag injection**: an edge function that sits in front of `index.html`, looks at the request URL, fetches the relevant data from your BaaS, and rewrites the `<head>` of the HTML response with per-route `<title>`, `<meta name="description">`, and Open Graph tags before serving. Crawlers see fully-rendered, route-specific metadata. Real users see the same SPA they always saw, because the HTML body is unchanged.

A minimal Netlify Edge Function example:

```
// netlify/edge-functions/inject-meta.ts
import type { Context } from 'https://edge.netlify.com'

export default async (request: Request, context: Context) => {
  const url = new URL(request.url)
  const response = await context.next()

  // Only inject for HTML responses
  if (!response.headers.get('content-type')?.includes('text/html')) {
    return response
  }

  // Determine route metadata
  const meta = await getMetaForRoute(url.pathname)  // fetch from your BaaS
  if (!meta) return response

  let html = await response.text()
  html = html
    .replace(/<title>.*?<\/title>/, `<title>${escapeHtml(meta.title)}</title>`)
    .replace(/<meta name="description"[^>]*>/, `<meta name="description" content="${escapeHtml(meta.des
    .replace('</head>', `
      <meta property="og:title" content="${escapeHtml(meta.title)}">
      <meta property="og:description" content="${escapeHtml(meta.description)}">
      <meta property="og:image" content="${meta.ogImage}">
```

```
      <meta property="og:url" content="${url.href}">
      </head>`)

  return new Response(html, {
    status: response.status,
    headers: response.headers,
  })
}

export const config = { path: '/*' }
```

Pair this with a `sitemap.xml` generator (also an edge function or a build-time script) that lists every entity/route the BaaS knows about, and submit the sitemap to search engines from the platform dashboard.

> [WARNING] **Sandbox restrictions on edge functions are real and undocumented.** Netlify Edge Functions, Cloudflare Workers, and Vercel Edge Functions all run Deno or V8 isolates with stricter syscall surfaces than `deno run` locally. `Deno.readFile()` is blocked in Netlify's sandbox, so loading fonts or static assets from the function's own filesystem will fail silently or with an opaque error. **Fetch fonts and static assets from a CDN URL at runtime instead.** See the *Netlify Edge Function sandbox restrictions* gotcha in Appendix A. The case-study Track B project (ABM, v8.5) burned a release on this when their OG image generation pipeline tried to read font files from disk and crashed with no error message.

> [TIP] **Static fallback first, dynamic later.** If you don't need per-route meta yet, set good *static* fallback meta tags in the actual `index.html` and skip the edge function entirely. Static OG tags cover ~80% of the value (the home page link looks right when shared, the project ranks in search) for ~5% of the work. Add the edge function later, when you have specific routes that need their own preview cards. The Track B case study did exactly this — static fallback in v8.0, edge-injected per-route meta in a later release.

[OK] Track B Phase 7 verification

- ☐ The footer on the live site displays the real version + git SHA matching the latest commit on `main`
- ☐ `curl https://your-domain.com/ | grep "rev <sha>"` succeeds for the latest deploy
- ☐ Sentry is receiving events tagged with the current release
- ☐ Lighthouse CI reports are uploaded as artifacts and accessible from CI runs
- ☐ UptimeRobot is monitoring both the public URL and the BaaS status page
- ☐ You have a written visual QA matrix template in your operations runbook (Phase 8 / Appendix B)
- ☐ At least one full release has gone through the visual QA matrix and the table is committed to the retro

Track B Phase 8: Documentation

> *Identical to Track A Phase 8.* `docs/CI-CD.md`, `docs/OPERATIONS-RUNBOOK.md`, retrospectives per release, `CHANGELOG.md` from release-please, in-repo `CLAUDE.md`. Same structure, same rules. The content describes Track B mechanics instead of Track A mechanics, but the discipline is the same.

The one delta worth naming: the operations runbook for Track B is shorter, because most of what would be in a Track A runbook is "how to use the platform's dashboard." A typical Track B runbook covers:

- **Manual deploy:** the CLI command (`netlify deploy --prod ...`) for when CI is broken
- **Rollback:** the dashboard URL + the `netlify rollback` CLI command
- **BaaS migration failed in CI:** how to inspect the failure, roll back the migration if needed, and re-trigger
- **Visual QA matrix:** the template (page × theme × viewport) with instructions for filling it in
- **Sentry alerts:** where they go, how to triage, what counts as urgent

- **Where the secrets live:** GitHub Actions secrets, Netlify environment variables, Supabase dashboard, Sentry project settings — name each one and what it's for

  Field note — from the case-study Track B project: *Three projects, three independent convergences on the same retro practice.* The case-study Track B project (Horomancy) ships with `docs/WORKFLOW.md`, a codified workflow document the team wrote on their own — versioned backlog, narrative retrospectives per release, themed milestones, dual-deploy ordering, quality gates with visual QA. None of it was copied from this playbook. The team arrived at the same shape from first principles. When three differently-built projects independently reach for the same tool, the tool is load-bearing.

[OK] Track B Phase 8 verification

- ☐ `docs/CI-CD.md` exists and describes your specific CI workflow, deploy job, BaaS migration step, and Sentry release tagging
- ☐ `docs/OPERATIONS-RUNBOOK.md` exists with manual deploy, rollback, BaaS migration recovery, visual QA matrix template, and secrets inventory
- ☐ `docs/retrospectives/` has at least one entry written in your own voice
- ☐ `CHANGELOG.md` is auto-generated by release-please
- ☐ `CLAUDE.md` (or equivalent) at the repo root describes the project, key architectural decisions, common commands, and where the secrets live

Track B summary

You have, in roughly the same number of phases as Track A and dramatically fewer lines of infrastructure code:

- A frontend that builds with Vite and stamps version + git SHA into the bundle
- A CI pipeline that lints itself, type-checks, tests, builds, runs Playwright E2E on desktop + mobile profiles with axe-core accessibility assertions, and runs Lighthouse CI with performance and accessibility budgets
- A deploy job that runs BaaS migrations, then atomic-deploys the artifact via the platform CLI, with post-deploy curl verification of the live SHA
- release-please handling automated version bumping in `package.json`
- Slack notifications on success and failure via Block Kit
- Client-side Sentry capturing render errors with release attribution
- Lighthouse CI providing longitudinal performance and accessibility observability
- A visual QA matrix as the post-deploy smoke test, with zero console errors as a hard gate
- Documentation (CI-CD, runbook, retros, CHANGELOG, CLAUDE.md) covering all of the above

The pipeline is boring. The platform handles atomicity, blue-green, rollback, SSL renewal, OS updates, CDN propagation, and most of nginx for you. Your job is to make sure the gates catch what they need to catch, the visual QA matrix gets walked on every release, and the retros get written.

Two Track B case studies ground this section

The patterns above are not extracted from a single project's quirks. **Two case studies of different shapes and tempos sit behind the Track B section**, and where their lessons disagreed they were reconciled inline.

- **Horomancy** is the velocity exemplar — a time-tracking web app that shipped nine major versions in three days from scaffold to PWA. Heavy parallel-agent collaboration, tight feedback loops, narrative retros after every release. Most of the Track B *structural* patterns (atomic platform deploy, version stamping via Vite `define:`, the visual QA matrix, mobile-profile Playwright, Lighthouse budgets, client-side Sentry with `ErrorBoundary`) trace to Horomancy because the project hit them first and most cleanly.

- **Anything But Metric (ABM)** is the depth exemplar — a unit-comparison web app shipped across twelve major releases over a longer arc, with explicit security audit passes (v8.1 and v9.0) and an explicit visual audit pass (v12). Most of the Track B *security and edge function* patterns (the handler pattern, the default-to-anon rule, the pre-deployment configuration ordering, SPA SEO via meta-tag injection, the `app_metadata.role` admin check rule, the PostgREST `.or()` injection gotcha, the Netlify Edge Function sandbox restrictions) trace to ABM because the project shipped longer, hit more edge cases, and ran the audits that surfaced them.

Both projects independently developed the same workflow discipline — themed releases, narrative retros, dual-deploy ordering, quality gates with visual QA, codified workflow documents — without copying from each other or from this playbook. The patterns in Track B are what *survived* both projects: the things that were load-bearing in two different teams, two different velocities, two different surface areas.

The velocity isn't the point. The depth isn't the point. The *convergence* is the point — when two differently-shaped projects independently reach for the same pattern, the pattern is real.

Appendix A: The Gotcha Hall of Fame

These are the things that bit us. They will bite you too. This list is your head start.

Severity legend — each gotcha is tagged so you know which ones to read first:

- [RED] **Will ship you** — production-breaking, common, hard to debug. Read these before you touch the relevant phase.
- [ORANGE] **Will burn an afternoon** — confusing, time-sink, but recoverable. Expect to meet these.
- [YELLOW] **Good to know** — edge case or environment-specific. File them away; they'll matter on the one project where they do.

Numbers are stable — they're referenced elsewhere in the playbook. Don't renumber.

1. [ORANGE] GitHub default branch vs. git-flow

If you're using `develop` + `main` git-flow, GitHub's default branch must be `develop`. Otherwise release-please and various other tools default to the wrong branch. Set it in Settings → Branches BEFORE you start wiring up release-please.

2. [ORANGE] Docker build context is everything in the build dir

A default `.dockerignore` doesn't exclude enough. Audit yours regularly. Use the "Verify build context exclusions" CI step from Phase 3 to prevent regression.

3. [ORANGE] `target-branch` in release-please-config.json is silently ignored

release-please v16+ takes `target-branch` as an **action input** in the workflow YAML, not as a root-level key in the config file. The config file silently accepts it (no schema validation) but the value is never used. If you find release-please opening PRs against `develop` instead of `main`, this is almost certainly why.

Fix: set `target-branch: main` in the `with:` block of the release-please-action step.

4. [ORANGE] release-please needs a GitHub Release anchor

With no prior Release, release-please replays commit history back to the beginning of the repo, which often surfaces old `BREAKING CHANGES:` markers and produces wildly wrong version proposals (the case-study project got proposed `6.0.0` when it expected `5.11.0`, because release-please had walked back to a `BREAKING CHANGES:` footer from a year before that the team had long since forgotten about). `bootstrap-sha` in the config is supposed to anchor this, but in practice (release-please 16.10.0) the short-SHA form didn't work for us.

Workarounds:

- Use a full 40-character SHA for `bootstrap-sha`
- OR manually create an anchor GitHub Release with the `gh release create` command
- OR use a one-shot `release-as: X.Y.Z` in the config to force the next version, then remove it immediately after ship

5. [RED] `GITHUB_TOKEN`-created releases don't trigger downstream workflows

This is intentional GitHub security — it prevents workflow loops. But it means release-please running with `GITHUB_TOKEN` creates a real GitHub Release that never fires `release: [published]`, so your `deploy.yml` never runs.

Fix: create a Personal Access Token with `contents: write` scope, add it as the `RELEASE_PAT` secret, and pass `token: ${{ secrets.RELEASE_PAT }}` to the release-please action. Releases created by a PAT DO fire downstream workflow triggers.

Workaround if you haven't set up the PAT yet: dispatch `deploy-only.yml` manually via `workflow_dispatch` after the GitHub Release appears.

6. [YELLOW] Slack `mrkdwn` is not standard Markdown

`##` headers, `-` bullets, blockquotes — all render as literal text in Slack. Use Block Kit JSON with `header` / `section` / `context` / `fields` blocks and `*bold*` for emphasis. See Phase 6.

7. [RED] `docker compose --env-file` doesn't reliably propagate to build args

Compose's `${GIT_REV:-unknown}` substitution for build arg values was inconsistent in the case-study project's testing — we never fully diagnosed why, but the bug is reproducible. The reliable fix is to pass `--build-arg GIT_REV=...` directly on the `docker compose build` command. This is the docker-level equivalent of `docker build --build-arg` and cannot be shadowed by any YAML substitution issue.

8. [RED] `/health` vs `/api/v1/health`

Have two endpoints. `/health` is lightweight status for nginx/load balancers. `/api/v1/health` is the full authenticated version with `git_sha`. **Your deploy verification must hit the authenticated one** and pass the API key + forwarded headers your middleware requires. See Phase 7.1. *(The Engineer + Agent Playbook Part V field note "The Health Check That Wasn't" tells the collaboration-side story: three days of agent-led fixes on a hole that wasn't there.)*

9. [RED] Bash self-reexec after git pull

When your deploy script does `git pull origin main`, any subsequent changes to the script itself are on disk but NOT in the running bash process's memory. Bash buffers scripts. Without `exec bash "$0"` after the pull, you're still running the OLD script.

```
if [ -z "${DEPLOY_REEXEC:-}" ]; then
    export DEPLOY_REEXEC=1
    exec bash "$0"
fi
```

10. [ORANGE] Commitlint `subject-case` rule

`@commitlint/config-conventional` disallows sentence-case subjects by default. Your subjects must start with a lowercase letter. Don't paste issue IDs at the start — put them in parens at the end:

- ☒ `fix(ci): JIRA-153 target main branch`
- [OK] `fix(ci): target main branch (JIRA-153)`

11. [YELLOW] `sed` delimiter injection on user input

Any workflow that takes a `workflow_dispatch` input and passes it into a `sed` substitution is vulnerable to the user typing a character that matches the sed delimiter. Pipe character is the classic — `inputs.reason="database issue | hotfix"` will break `sed "s|REPLACE|${{ inputs.reason }}|"`. Use `jq -n --arg` for any JSON construction involving user input.

12. [YELLOW] Squash-merge divergence

Squash-merging a feature branch into develop, then merging develop to main, then making main-only fix commits on separate branches off main, creates a reconciliation debt on develop. The next `git merge main` into develop will conflict on files both sides touched (even though the changes are conceptually compatible). Resolve by taking main's version wholesale; this is git-flow's known cost of squash.

13. [ORANGE] "Evil merges"

Git's 3-way merge can produce surprising results where a line removed on one side silently reappears on the other side after merge, because the base commit didn't have the line at all. Always review the merged result line-by-line before committing a merge. The case-study project hit this with a one-shot `release-as: X.Y.Z` line in its release-please config that had been deliberately removed on `main`, then silently reappeared after a `main`-to-`develop` merge — and would have caused the next release to redo a version bump it had already shipped.

14. [YELLOW] Hyphens in wordlists become phrase separators

If your code does `"-".join(random.choices(words, k=3))` and the wordlist contains hyphenated words like `"drop-down"`, your "three-word phrase" splits into 4 parts instead of 3. Audit any wordlist used for string composition where the join character also appears inside the items. The case-study project had 4 hyphenated words (`drop-down`, `felt-tip`, `t-shirt`, `yo-yo`) in its random-phrase wordlist, causing ~0.15% of generations to fail downstream validation — silently handled by a retry loop for months until somebody actually counted the retries.

15. [ORANGE] `bash -c` + `export VAR=$(...)` + `set -e` interactions

`set -e` does NOT abort on failures in command substitution inside variable assignment. `FOO=$(failing-command)` will succeed if the assignment itself succeeds, even if the subshell failed. Use explicit checks: `FOO=$(cmd) || exit 1`.

16. [ORANGE] Vite + `import.meta.env.npm_package_version` is unreliable *(Track B)*

If you stamp the version into a Vite-built frontend by reading `process.env.npm_package_version` or `import.meta.env.npm_package_version`, the value depends on which npm version invoked the build *and* on the surrounding Vite configuration. The case-study Track B project shipped a release where the live footer displayed `v1.0.0` long after the project was on v5.0 — because `npm_package_version` returned the wrong value depending on how `vite build` was invoked.

Fix: read `package.json` directly with `readFileSync` at config-evaluation time, and inject via Vite's `define:`. This is deterministic and always works.

```
// vite.config.ts
import { readFileSync } from 'fs'
import path from 'path'
import { fileURLToPath } from 'url'

const __dirname = path.dirname(fileURLToPath(import.meta.url))
const pkg = JSON.parse(readFileSync(path.resolve(__dirname, 'package.json'), 'utf-8'))
```

```
export default defineConfig({
  define: {
    __APP_VERSION__: JSON.stringify(pkg.version),
  },
})
```

This is the Track B equivalent of Track A's `--build-arg GIT_REV` pattern (Gotcha #7) — same lesson, different mechanism: **never rely on environment-variable propagation for build-time version stamping; read the source of truth directly.**

17. [RED] Publishable vs. service-role keys on BaaS *(Track B)*

BaaS systems ship with two kinds of keys, and conflating them is how data gets leaked. Using Supabase as the example:

- **`anon` key (publishable)** — client-safe by design. Goes in the client bundle. Protected by Row-Level Security policies on every table. If you don't have RLS configured, the anon key gives unrestricted read/write access to your entire database. **RLS is your authorization boundary; the anon key is just an API entry token.**
- **`service_role` key (secret)** — bypasses RLS entirely. Has full admin access to every table. Must NEVER appear in client code, CI logs, the git repo, or anywhere a user could see it. It belongs in server-side environments (CI secrets, BaaS dashboard env vars for cloud functions) and nowhere else.

Other BaaS systems have the same pattern under different names: Firebase has `apiKey` (publishable) vs. service account credentials (secret). Clerk has the publishable key vs. the secret key, named exactly that. PlanetScale has connection strings with branch-scoped vs. admin permissions.

Common mistakes to never make:

- Committing the service role key to the repo "just for local dev." Local dev uses `supabase start` (or your BaaS equivalent), which generates its own local keys that are safe by definition.
- Using the service role key in the client bundle "to bypass RLS for one query." If you need to bypass RLS, the right answer is a `SECURITY DEFINER` RPC function, not a leaked key.
- Storing the service role key in GitHub Actions secrets without restricting which workflows can read it. Limit access to the deploy job only.
- Logging the env vars in CI for debugging. Never. The CI logs are part of your audit trail and will leak the key into screenshots, retros, and future searches.

If you're seeing a key in a client bundle and panicking — first check whether it's the publishable key (in which case you're fine, RLS protects you) or the service role key (in which case you have an active incident, rotate immediately).

18. [RED] Default-to-admin in BaaS handler signatures *(Track B)*

A shared handler function that takes a `client:` parameter must default to the *anon* client, not the admin client. The case-study Track B project shipped a refactor where the default was `'admin'`, and any new endpoint written without thinking about the field silently inherited admin privileges. Two endpoints — a public stats endpoint and a public leaderboard — ran with admin keys for a full release cycle. RLS happened to cover the data they touched, so no leak occurred, but the privilege gradient was inverted. The fix was one line: change the default. The lesson generalizes to *every* security-relevant default in *every* shared handler: the unsafe choice should not be the one developers (and agents) get when they don't think about the field. See Track B Phase 4.7.

19. [RED] PostgREST `.or()` is an injection vector with user input *(Track B)*

PostgREST's `.or()` query builder accepts a string of comma-separated filter expressions. Passing user input into that string — entity names containing commas, dots, parentheses, the literal word `and` — opens an injection path that lets a caller break out of the intended filter and read or alter rows the original query

never meant to touch. The case-study Track B project hit this with entity names like `"3.5"` and `"foo,bar"` that broke the parser in surprising ways.

Fix: never interpolate user input into `.or()` strings. Use multiple parameterized `.eq()` queries in parallel and merge the results client-side, or use a database function with explicit parameters. The pattern applies to every PostgREST-style query builder; the same trap exists in PostgREST itself, in Supabase's JS client, in PostGREST-on-Postgrest, and in any framework that builds query strings from user input.

20. [RED] Supabase admin checks must read `app_metadata.role`, not `user_metadata.role` *(Track B)*

Supabase Auth stores user attributes in two JWT fields: `app_metadata` (writable only by the service role, i.e., your backend) and `user_metadata` (writable by the user themselves, via `supabase.auth.updateUser({ data: ... })`). An admin check that reads `user_metadata.role` lets *any user grant themselves admin privileges* by editing their own profile. The check must read `app_metadata.role`, which only the service role can write.

```
// WRONG - user-writable, trivially escalatable
const isAdmin = session.user.user_metadata?.role === 'admin'

// RIGHT - only the service role can write this field
const isAdmin = session.user.app_metadata?.role === 'admin'
```

This is the single most common Supabase RBAC mistake and it ships in production code regularly. The case-study Track B project caught it in a v8.1 security audit; the fix was a one-line code change with privilege-escalation consequences if it had stayed in place. **RLS policies have the same trap** — `auth.jwt() ->> 'role'` reads the JWT top-level role, not user_metadata, but a poorly-written policy that joins to `auth.users.raw_user_meta_data` is just as exploitable. Audit every admin check in your codebase the day you add the first one.

21. [ORANGE] The Guard That Came Before the Secret *(both tracks, but bites Track B more)*

A deploy that adds a startup-time check requiring some configuration to exist (a secret, an env var, a config file, a database row) must include the step that creates the prerequisite — *and* must execute it *before* the deploy that depends on it lands. The case-study Track B project shipped a v8.3 release that added a startup check for `IP_SALT`. The PR was clean, tests passed, code review approved it. The deploy ran. The new container started. The startup check fired. The container exited. Auto-rollback caught it after two retries. Root cause: nobody had added `IP_SALT` to the production secrets vault. Fix was thirty seconds; outage was thirty minutes.

Lesson: when a deploy introduces a precondition, the deploy plan must include the step that creates the precondition's prerequisite, in the right order, in the right system. Walk the plan against this checklist before any deploy that touches more than the application code:

1. New startup check? → is the thing the check requires already in place in the target environment?
2. New secret/env var/config reference? → is it in the secrets vault / dashboard?
3. New database column/table/function reference? → has the migration run, *before* the application code lands?
4. New third-party resource reference? → registered with the third party?
5. New contract with another system? → backwards-compatible or scheduled in lockstep?

See Track B Phase 4.8 for the full discussion. Bites Track B harder than Track A because the platform's atomic deploy gives you fewer recovery options between "deploy started" and "deploy completed" than a self-managed blue-green flip does.

22. [YELLOW] Supabase edge function deployment latency *(Track B)*

`supabase functions deploy` reports success when the new code has been *uploaded*, not when it's actually live. There's a propagation window — usually a few seconds, occasionally longer — during which a request to the function can still hit the previous version. A post-deploy verification that hits the function immediately can pass against the *old* build and give you a green check that means nothing.

Fix: include a marker in the new build (a version constant, a git SHA in a header, a build timestamp in a JSON response) and have your verification curl wait until the response includes the new marker before declaring the deploy verified. Same pattern as Track A's "post-flip smoke test must hit the public URL through nginx, not the container directly" — the principle is *verify against the same surface a real user would hit, after the same propagation a real user would experience.*

23. [YELLOW] `deno.lock` v5 vs Supabase CLI version mismatch *(Track B)*

The case-study Track B project's developers regenerated `deno.lock` locally with a newer Deno toolchain. The new file was a v5 lockfile. The Supabase CLI version on CI only understood v4 lockfiles. Deploy failed mid-pipeline with a confusing error that didn't mention the lockfile version. The fix was to either pin Deno locally to the version CI expected, or upgrade CI to a Supabase CLI version that handled v5.

Lesson: toolchain version drift between local and CI is the quiet killer. It applies to every toolchain — Deno, Node, Python, Rust, Go, the lockfile formats they each have, the CLIs that consume them. Pin the toolchain version explicitly in CI (`actions/setup-node` with a fixed version, `actions/setup-python` with a fixed version, `denoland/setup-deno` with a fixed version), pin it locally too (`.nvmrc`, `.python-version`, `rust-toolchain.toml`), and make CI fail loudly if the local toolchain doesn't match the pinned version. Same root cause as Gotcha #7 (compose env-file vs `--build-arg`) — a tooling version mismatch invisible until it bites.

24. [ORANGE] Netlify Edge Function sandbox restrictions *(Track B)*

Netlify Edge Functions run in a Deno sandbox that's stricter than `deno run` locally. The most painful restriction: **`Deno.readFile()` is blocked.** Code that loads font files, static assets, JSON config, or any other on-disk resource from the function's own filesystem will fail in production with no error message — the function just crashes silently or returns an opaque 500. The case-study Track B project (ABM, v8.5) burned a release on this when their OG image generation pipeline tried to load font files from disk for the SVG-to-PNG render step.

Fix: fetch fonts and static assets from a CDN URL at runtime instead of bundling them with the function. The Track B Phase 7.6 SPA SEO subsection has the full pattern. Same restriction applies to Cloudflare Workers (V8 isolates with no filesystem) and Vercel Edge Functions (similar Deno sandbox). **Test in the actual sandbox before declaring done** — what works in `deno run` locally is not what runs in production.

25. [YELLOW] Vitest version sensitivity to Node version *(Track B, but applies anywhere Vitest runs)*

The case-study Track B project's CI pipeline got slower — *much* slower. 159 unit tests went from running in 7 seconds to running in 7 minutes. The team initially assumed it was new tests, then assumed it was a CI runner change, then finally diagnosed the actual cause: Vitest 1.6.1 had a 57× slowdown on Node 25 due to a worker-pool regression. Upgrading Vitest to a later version restored the previous performance.

Lesson: your test runner has version sensitivities; pin both the runner version and the Node major version, and audit them when CI starts feeling slow. A test suite that suddenly takes 60× longer is a signal that something underneath it has changed even if your test code didn't. The first place to look is the runner-vs-runtime compatibility matrix, *before* you start refactoring tests. The wider lesson: any tool you use across hundreds of CI runs accumulates subtle version-compatibility surprises; the moment something feels slower than it used to, audit the toolchain versions before you audit the work.

Appendix B: Ship-Readiness Checklist

Before you call it done, walk this list. Every box should be checked.

Foundation

- ☐ `develop` and `main` branches exist, `develop` is the default
- ☐ `main` has branch protection (require PR, require status checks)
- ☐ `.gitignore` excludes `.env`, `__pycache__`, `node_modules`, `*.log`, `logs/`, etc.
- ☐ `.dockerignore` is aggressive, verified by the CI build context check
- ☐ `VERSION` file exists at repo root
- ☐ `commitlint.config.js` enforces conventional commits
- ☐ Commits are actually conventional (check last 10)

Local dev

- ☐ `docker compose up` starts the full stack locally
- ☐ `docker compose down` cleans up without errors
- ☐ Local `/health` responds
- ☐ `.env.example` is committed; `.env` is NOT

CI

- ☐ All actions are SHA-pinned (not `@v4` or `@main`)
- ☐ `workflow-lint` (actionlint) is the FIRST job and passes
- ☐ `commitlint` runs on PRs
- ☐ Backend tests run against real PostgreSQL/Redis (not mocks)
- ☐ Frontend tests include bundle size check
- ☐ `pip-audit` and `npm audit` enforce CRITICAL vulnerabilities
- ☐ `trufflehog` scans the git history for secrets
- ☐ Docker build validates exclusions (`.git`, `node_modules` not in image)
- ☐ Trivy scans the final image for CRITICAL+HIGH vulnerabilities
- ☐ CI failure on `main` notifies Slack

Deploy

- ☐ `deploy/deploy-bluegreen.sh` exists and has the flock lock
- ☐ `deploy/smoke-test.sh` exists and creates+reads a real resource
- ☐ `deploy/preflight.sh` exists and checks disk/docker/env
- ☐ `nginx/upstream-active.conf` exists with blue as default active
- ☐ Self-reexec pattern is in deploy-bluegreen.sh after git pull
- ☐ `GIT_REV` is passed via `--build-arg`, NOT just env file
- ☐ Manifest JSON is validated with `python3 -m json.tool` after write
- ☐ Post-deploy verification hits `/api/v1/health` with headers, NOT `/health`
- ☐ Auto-rollback path is tested (break smoke test, deploy, confirm rollback)

Release

- ☐ `release-please-config.json` exists WITHOUT `target-branch` at root
- ☐ `.github/workflows/release.yml` passes `target-branch: main` as action input
- ☐ release-please uses a PAT (`secrets.RELEASE_PAT`), NOT `GITHUB_TOKEN`
- ☐ GitHub repo setting "Allow GitHub Actions to create and approve PRs" is enabled
- ☐ `deploy.yml` triggers on `release: [published]`
- ☐ `deploy-only.yml` exists for emergency manual dispatch
- ☐ End-to-end test: feature commit → develop → main → Release PR → merge → GitHub Release → deploy → Slack messages

Observability

- ☐ `/health` returns `{"status":"ok"}` in <100ms
- ☐ `/api/v1/health` returns full JSON with real `git_sha`
- ☐ Frontend footer shows real version and git SHA
- ☐ UptimeRobot or equivalent monitors `/health`
- ☐ Sentry (or equivalent) is configured with `release=<git_sha>`

Docs

- ☐ `docs/CI-CD.md` exists and is accurate
- ☐ `docs/OPERATIONS-RUNBOOK.md` exists with manual deploy, rollback, troubleshooting
- ☐ Every file referenced in the runbook actually exists
- ☐ At least one retrospective is written

Gotchas

- ☐ You've read Appendix A once
- ☐ You have a PAT set up for release-please (Gotcha #5)
- ☐ Your deploy script self-reexecs (Gotcha #9)
- ☐ Your verification curl hits `/api/v1/health` (Gotcha #8)

Track B additions

If you're on Track B, also walk this list:

- ☐ `package.json` is the version source of truth (no separate `VERSION` file needed)
- ☐ `vite.config.ts` reads `package.json` with `readFileSync` for version stamping (Gotcha #16)
- ☐ Footer displays real `__APP_VERSION__` and `__APP_GIT_REV__` from `define:`
- ☐ BaaS local emulation works (`supabase start` or equivalent) and matches production schema
- ☐ `.env.local` contains the publishable anon key only; service role key is in CI secrets and BaaS dashboard, never in repo (Gotcha #17)
- ☐ `netlify.toml` (or platform equivalent) sets X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, CSP
- ☐ Lighthouse CI runs on every push with numeric budgets for performance and accessibility (Phase 2.4a)
- ☐ Playwright config includes at least one mobile/touch device profile (Phase 2.4c)
- ☐ axe-core accessibility tests run as part of E2E (Phase 2.4b)
- ☐ Deploy job runs BaaS migrations BEFORE platform deploy (Track B Phase 4.2)
- ☐ Post-deploy verification curls the live URL and confirms the footer git SHA matches the commit
- ☐ Client-side Sentry is wired with `ErrorBoundary → captureException` and release tag = `version@git_rev`
- ☐ UptimeRobot monitors both the public URL and the BaaS status page
- ☐ You have walked the visual QA matrix (below) on at least one real release

Visual QA matrix template

Applies to both tracks. Walk this matrix as the post-deploy smoke test. Track A readers run it in addition to the curl-based verification; Track B readers run it as the primary verification. Adapt the page list to your project; keep the columns.

Page	Dark	Light	Mobile (375px)	Notes
Landing / Login				
Signup				
Reset Password				
Main app view				

Page	Dark	Light	Mobile (375px)	Notes
Secondary view				
Settings / Account				
Modal flows				
Error states				

Hard gates (every release):

- ☐ Zero console errors across all pages
- ☐ Zero failed network requests in the network panel (excluding intentional 401/403 on protected routes)
- ☐ All security headers present (`curl -sI https://your-domain.com/` and verify CSP, X-Frame-Options, Referrer-Policy)
- ☐ One full end-to-end real interaction tested in production (sign up → core action → sign out)
- ☐ Footer git SHA matches the commit you just shipped
- ☐ No hover-only affordances are missing on the mobile/touch profile

Commit the filled-in matrix to the release retro. Future-you wants to know exactly which pages were verified on which deploy.

Appendix C: Common Commands Cheat Sheet

Development

```
# Start local dev stack
docker compose up -d --build

# Tail backend logs
docker compose logs -f backend

# Run backend tests
docker compose exec backend python -m pytest tests/

# Run frontend tests
cd frontend && npm run test

# Shut down
docker compose down

# Full reset (destroys db data)
docker compose down -v
```

Git

```
# New feature
git checkout develop
git pull
git checkout -b feat/my-feature
# ... work ...
git commit -m "feat: add thing"
git push -u origin feat/my-feature
gh pr create --base develop

# Release (after feature PR merged)
```

```
git checkout develop
git pull
git checkout main
git merge develop
git push origin main
# release-please opens a PR; merge it; deploy fires automatically
```

Deploy & operations

```
# Manual deploy on the server
ssh user@your-host
cd /opt/myapp
sudo bash deploy/deploy-bluegreen.sh

# Manual rollback
sudo bash scripts/rollback-bluegreen.sh

# Check what's deployed
curl -H "X-API-Key: $KEY" https://your-domain.com/api/v1/health | jq .git_sha

# Check deploy manifest on server
cat /opt/myapp/deploy-manifest.json

# Force-trigger deploy.yml (emergency path)
gh workflow run deploy-only.yml -f reason="your reason" --ref main

# Tail deploy logs from CI
gh run watch
```

Debugging

```
# Who changed what recently
git log --oneline -20

# What's different between develop and main
git log --oneline main..develop

# Find a specific change
git log --all --oneline -- path/to/file
git blame path/to/file

# Run actionlint locally before pushing
curl -sSL https://github.com/rhysd/actionlint/releases/download/v1.7.7/actionlint_1.7.7_$(uname -s | tr
/tmp/actionlint .github/workflows/

# Validate a compose file
docker compose -f docker-compose.prod.yml config

# Check a YAML workflow file parses
python3 -c "import yaml; yaml.safe_load(open('.github/workflows/ci.yml'))"
```

Closing

If you worked through every phase, you now have:

- A repo with protected branches and conventional commits
- Docker images that don't contain `.git`
- CI that tests, audits, scans, and lints itself
- Blue-green deploys with automatic rollback
- Fully automated releases (push to main → Release PR → GitHub Release → deploy → Slack)
- A manifest file on the server that tells you exactly what's running
- Health endpoints that don't lie about what commit is deployed
- Docs that explain it all and a runbook for when it's on fire
- A head start on the gotchas that cost us days to find

You also have a pipeline that will **catch bugs you didn't know you had** the first time it runs. Trust the walls. If `actionlint` finds 10 `SC2086` issues or `npm audit` finds a CVE or the context-exclusion check flags a stray directory — that's not noise. That's the pipeline paying for itself.

The last rule is the most important:

> **You are done when the pipeline is boring.**
>
> When it runs on every push and you don't even notice. When releases are one click and zero stress. When you can look away for a week and come back to find everything still green. When new engineers can ship their first feature without anyone walking them through deploys. When production failures become interesting because they're rare, not exhausting because they're constant.
>
> Getting there takes a weekend of focused work and two or three real incidents to learn the gotchas from. This playbook is those gotchas, pre-paid.

Good luck. Go build something.